

Semáforos Inteligentes: Un enfoque con Aprendizaje por Refuerzo Multi Agente

Tesina de Licenciatura en Ciencias de la
Computación

Juan Martín Morales

Tutora: Dra Ana Carolina Olivera



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



FACULTAD DE
INGENIERÍA

Facultad de Ingeniería
Universidad Nacional de Cuyo

Resumen

En este trabajo se aborda el problema de la congestión vehicular en áreas urbanas mediante un sistema de control semafórico basado en Aprendizaje por Refuerzo Multi-Agente (MARL) cooperativo. Se analiza la evolución del parque automotor en Argentina, se describen las limitaciones del control semafórico tradicional y se formula el problema como una tarea de decisión secuencial en la que las intersecciones actúan como agentes. El objetivo es desarrollar y evaluar políticas coordinadas que reduzcan las demoras y mejoren las condiciones de operación de la red. Para ello, se emplean simulaciones microscópicas en escenarios sintéticos y en una red vial que representa el microcentro de la Ciudad de Mendoza, y se comparan algoritmos independientes y cooperativos mediante métricas de tránsito y de aprendizaje.

Abstract

This work addresses urban traffic congestion through a traffic-signal control system based on cooperative Multi-Agent Reinforcement Learning (MARL). It analyzes the growth of Argentina’s vehicle fleet, describes the limitations of traditional signal control, and formulates the problem as a sequential decision task in which intersections act as agents. The objective is to develop and evaluate coordinated policies that reduce delays and improve network operating conditions. Microscopic simulations are conducted in synthetic scenarios and in an urban road network representing downtown Mendoza City, comparing independent and cooperative algorithms using traffic and learning metrics.

Agradecimientos

Página de agradecimientos

Índice

1	Introducción	1
1.1	Motivación	3
2	Marco Teórico	4
2.1	Ingeniería de Tránsito y Simulación	4
2.1.1	Terminología de Tránsito	5
2.1.2	Fundamentos de la Dinámica Vehicular	7
2.1.3	Sistemas de Control de Tránsito	12
2.1.4	Simulación de Tránsito	14
2.2	Aprendizaje por Refuerzo (RL)	18
2.2.1	El Proceso de Decisión de Markov (MDP)	19
2.2.2	Métodos Basados en Valor y Basados en Política	20
2.2.3	Aprendizaje Profundo y Redes Neuronales	23
2.2.4	Aprendizaje por Refuerzo Profundo (DRL)	26
2.3	Aprendizaje por Refuerzo Multi-Agente (MARL)	32
2.3.1	Fundamentos Teóricos: Juegos Estocásticos, Equilibrio de Nash y Dec-POMDP	33
2.3.2	Desafíos Fundamentales en MARL	35
2.3.3	Paradigmas de Entrenamiento y Ejecución	36
2.3.4	MADRL y líneas de base	36
2.3.5	Multi-Agent Proximal Policy Optimization (MAPPO)	37
2.3.6	HAPPO	40
3	Metodología y Diseño Experimental	46
3.1	Enfoque y Diseño de Investigación	46
3.2	Especificación del Ecosistema Tecnológico	49
3.3	Diseño y Calibración de Escenarios de Tránsito en SUMO	50
3.3.1	Modelado Geométrico y Sanitización Topológica	50
3.3.2	Escenarios Sintéticos: 3×1 , 3×3 y $4 \times 4_H$	50
3.3.3	Escenario Urbano Real: Microcentro de la Ciudad de Mendoza	52
3.4	Arquitectura de Integración Entorno-Agentes	57

3.4.1	Ciclo Episódico e Interfaz de Simulación	57
3.5	Instanciación del Modelo Dec-POMDP	58
3.5.1	Espacio de Observaciones y Codificación Logarítmica	58
3.5.2	Espacio de Acciones y Enmascaramiento Dinámico	59
3.5.3	Función de Recompensa Compuesta y Coordinación Vecinal	59
3.6	Implementación Algorítmica e Hiperparámetros	60
3.6.1	Mecanismos de Aprendizaje Independiente y Centralizado	60
3.7	Búsqueda de Hiperparámetros (HPO)	60
3.7.1	Infraestructura de Cómputo y Orquestación	62
3.8	Evaluación de Rendimiento	62
4	Análisis y discusión de resultados	64
4.1	Resultados del ajuste de hiperparámetros	64
4.1.1	Diseño del proceso de ajuste	64
4.1.2	Resultados del ajuste y análisis de sensibilidad	64
4.1.3	Configuración seleccionada para la evaluación final	64
4.2	Resultados de los modelos finales	64
4.2.1	Criterios de evaluación y métricas de comparación	65
4.2.2	Desempeño en escenarios sintéticos: 3×1 , 3×3 , 4×4 heterogéneo	65
4.2.3	Desempeño en la red de carreteras de la Ciudad de Mendoza	65
4.3	Discusión e interpretación de resultados	65
5	Conclusiones	66
A	Arquitectura de Software y Patrones de Diseño	67
A.1	Diseño Orientado a Objetos del Entorno de Simulación	67
A.2	Jerarquía de Funciones de Recompensa y Patrón Strategy	68
A.3	Patrones de Diseño Implementados	69

Índice de figuras

1.1	Evolución de la flota vehicular circulante en Argentina (1990–2025). Datos: AFAC.	1
2.1	Movimientos vehiculares, fases y cruces peatonales	6
2.2	Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.	6
2.3	Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).	9
2.4	Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de pelotones. Las curvas representan situaciones conceptuales y no una distribución empírica particular.	12
2.5	Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.	13
2.6	Elementos geométricos y conexiones de una intersección semaforizada en SUMO bajo sentido de circulación por la derecha (Argentina): carriles entrantes (L_{in}), salientes (L_{out}), líneas de detención y enlaces de cruzamiento ($links$) en verde prioritario ($\mathbf{G}$) y rojo retenido en conflicto ($\mathbf{r}$).	15
2.7	Interacción agente-entorno en un proceso de decisión de Markov [SB18]. . .	19
2.8	Arquitectura actor-crítico aplicada al control semafórico. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].	22
2.9	Relación entre las familias de algoritmos utilizadas en esta investigación. Los cuatro métodos evaluados son libres de modelo y se derivan de las líneas DQN y PPO.	23
2.10	Operaciones de una unidad neuronal artificial: combinación afín de las entradas y aplicación de una función de activación.	24
2.11	Arquitectura conceptual de una red neuronal <i>feedforward</i> . Cada columna representa una capa y las conexiones se dirigen desde la entrada hacia la salida; las expresiones inferiores indican la transformación afín seguida de la activación en cada capa.	25

2.12	Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de <i>bootstrapping</i> del blanco TD. Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS ⁺ 15].	28
2.13	Red de política para un espacio discreto de fases. La observación determina una distribución de acciones; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03]. . .	30
2.14	Interacción agente-entorno en MARL cooperativo	35
2.15	Paradigma de Entrenamiento Centralizado y Ejecución Descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.	37
2.16	Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m} , que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.	43
3.1	Flujo metodológico secuencial de la investigación experimental.	48
A.1	Diagrama de clases UML de la infraestructura del entorno de simulación y controladores semafóricos.	68
A.2	Diagrama de clases UML de la jerarquía de funciones de recompensa (Patrones Strategy y Composite).	69

Índice de tablas

2.1	Variables de tránsito macroscópicas y microscópicas	9
2.2	Archivos de configuración empleados en los escenarios de simulación en SUMO [DLR25].	17
2.3	Comparación sintética de familias algorítmicas PPO en entornos multi-agente cooperativos.	45
3.1	Especificación de las tecnologías y herramientas utilizadas en la investigación.	49
3.2	Métricas operacionales de validación del escenario canónico v14 de Mendoza.	56

Índice de códigos

Glosario

Actor (Aprendizaje por Refuerzo) Componente de la arquitectura del modelo de aprendizaje responsable de seleccionar y ejecutar la acción en el entorno basándose en el estado actual y la política que está aprendiendo.

Agente Entidad computacional que observa el estado de su entorno y ejecuta acciones según una política determinada con el objetivo de maximizar su recompensa a largo plazo.

Ciclo (Ingeniería de Tránsito) Secuencia completa y repetitiva de todas las fases semafóricas programadas en una intersección particular.

Cola vehicular Fila de vehículos que se detienen o avanzan muy lentamente debido a una restricción en la capacidad de la vía, tal como la luz roja de un semáforo.

Crítico (Aprendizaje por Refuerzo) Componente encargado de estimar la función de valor, prediciendo la recompensa futura esperada para evaluar la calidad de las acciones tomadas por el Actor y así guiar su actualización.

Densidad vehicular Número de vehículos presentes en un tramo o longitud específica de un carril en un instante dado.

Entorno El sistema dinámico con el que interactúa el agente. Responde a las acciones del agente produciendo transiciones de estado y emitiendo señales de recompensa.

Estado Representación cuantitativa o cualitativa de la situación actual del entorno en un instante de tiempo específico. Sirve como base para que el agente seleccione su próxima acción.

Fase (Ingeniería de Tránsito) Estado temporal de un controlador semafórico que habilita el derecho de paso exclusivo a una o más corrientes vehiculares incompatibles para que crucen una intersección de forma segura.

Flujo vehicular Cantidad de vehículos que pasan por una sección transversal específica de un carril o calzada durante un intervalo de tiempo determinado.

Función de Valor Estimación matemática de la cantidad total de recompensa que un agente puede esperar acumular en el futuro, comenzando desde un estado particular y siguiendo una política específica.

Observación Subconjunto de información del estado completo del entorno que es efectivamente visible o medible por un agente individual, especialmente relevante en escenarios de observabilidad parcial.

Política (Aprendizaje por Refuerzo) Estrategia que define el comportamiento del agente en un momento dado, mapeando los estados observados a probabilidades de selección de las acciones disponibles.

Proceso de Decisión de Markov Marco matemático utilizado para modelar la toma de decisiones en situaciones donde los resultados son en parte aleatorios y en parte bajo el control de un agente tomador de decisiones.

Recompensa Señal escalar devuelta por el entorno que evalúa cuán buena o mala fue la acción ejecutada por el agente en un estado particular. El objetivo del agente es maximizar la suma total de estas señales.

Ventaja (Aprendizaje por Refuerzo) Métrica que cuantifica cuánto mejor o peor resulta tomar una acción particular en un estado dado, en comparación con el desempeño promedio esperado según la política actual.

Introducción

En las últimas décadas, en el mundo se ha producido un incremento sostenido del parque automotor a nivel mundial, acompañado por un proceso continuo de urbanización que ha convertido a la congestión vehicular en uno de los principales desafíos para la movilidad y eficiencia en áreas urbanas. Según los reportes de flota vehicular de Argentina, elaborados por la Asociación de Fábricas Argentinas de Componentes (AFAC), y utilizando el año 2012 como línea base, el volumen de vehículos registró un aumento del 21,7% en 2017. Dicha progresión se sostuvo a lo largo de la última década, alcanzando un incremento acumulado del 44,4% para el año 2025 (véase la Figura 1.1).

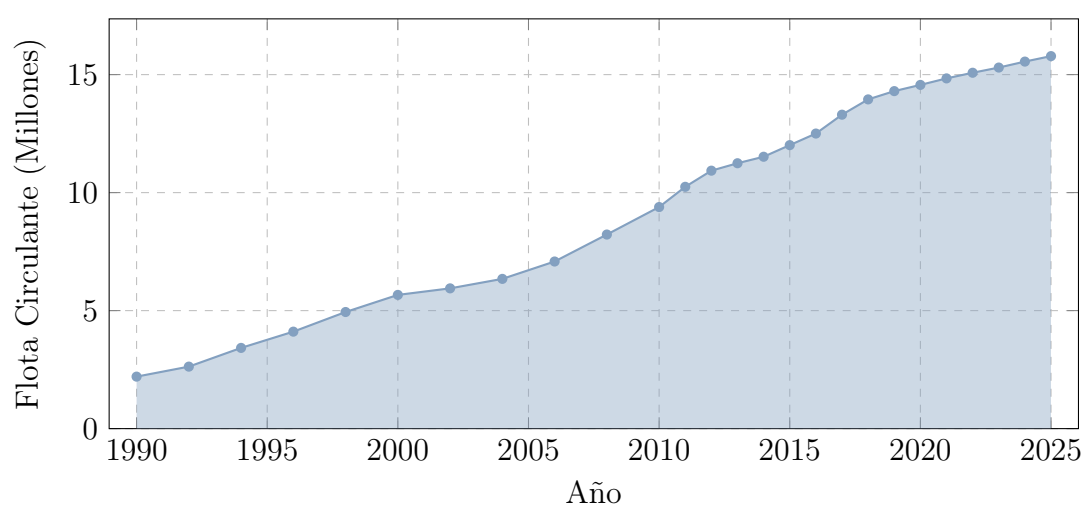


Figura 1.1: Evolución de la flota vehicular circulante en Argentina (1990–2025). Datos: AFAC.

Las congestiones vehiculares deterioran las condiciones de operación de la red e in-

crementan las demoras y las detenciones de los vehículos [CyMR18]. En el modelado microscópico, las emisiones de gases como CO, CO₂, NO_x y HC se estiman a partir de variables cinemáticas, entre ellas la velocidad, la aceleración y el tiempo en ralentí [DLR25]. Este escenario impacta de manera notable en grandes ciudades con una alta densidad de vehículos. En este trabajo se utilizaron casos de estudio sintetizados y, como caso de estudio urbano restringido, el microcentro de la Ciudad de Mendoza, delimitado por Avenida San Martín, Avenida Colón, Avenida Belgrano y Avenida Las Heras.

Frente a un escenario de congestión, una alternativa viable, sin recurrir a reestructuraciones costosas en la infraestructura, consiste en optimizar la gestión del tránsito mediante sistemas de control semafórico adaptativos. Los enfoques tradicionales de control semafórico han demostrado poca capacidad de adaptación ante escenarios con mucha variabilidad de flujos de vehículos. Una alternativa prometedora resulta ser el Aprendizaje por Refuerzo (*Reinforcement Learning*, RL), un paradigma de inteligencia artificial dónde los semáforos pueden modelarse como agentes, capaces de aprender políticas de control a partir de la interacción con un entorno (la situación del tránsito) mediante la ejecución de acciones (cambios de fase) con el objetivo de maximizar una recompensa (como, por ejemplo, el valor negativo de la suma de los tiempos de espera). Diversos estudios han demostrado la viabilidad de este enfoque, evolucionando desde la aplicación de algoritmos clásicos y tabulares en una sola intersección [Wie00], hasta modelos a gran escala de redes coordinadas empleando redes neuronales profundas y control multi-agente [W⁺19].

Sin embargo, aplicar este enfoque de manera simultánea sobre múltiples intersecciones introduce desafíos adicionales. Un esquema de control centralizado resulta poco escalable debido al crecimiento exponencial del espacio de acciones conjunto. Una alternativa “ingenua” del Aprendizaje por Refuerzo Multi-Agente resulta ser el Aprendizaje Independiente (*Independent Learning* o IL), donde cada semáforo busca descongestionar de forma independiente y egoísta solo su propia intersección [AS22]. Este enfoque tiene dos desventajas críticas: primero, la optimización puramente local conduce a un óptimo global sub-óptimo; segundo, dado que todos los agentes aprenden y cambian sus políticas simultáneamente, el entorno se vuelve no estacionario para cada agente individual, lo cual viola los supuestos de convergencia del aprendizaje [FH21].

Por esta razón, en este trabajo final de carrera se aborda el problema desde el marco del Aprendizaje por Refuerzo Multi-Agente (MARL) Cooperativo. Bajo este paradigma, los controladores (agentes) actúan como un equipo. Aprenden políticas coordinadas, no para maximizar solo una recompensa individual, sino también una recompensa global compartida orientada a la minimización del tiempo de espera promedio de toda la red, evitando así los conflictos que surgen del aprendizaje independiente. La recompensa es un valor numérico que indica cuán efectiva fue la acción conjunta.

En este contexto, el objetivo general de este trabajo fue desarrollar y evaluar un sistema de control de tráfico basado en este enfoque cooperativo para optimizar el flujo

vehicular urbano. Para lograrlo, se analizó la convergencia a políticas de control óptimas y la escalabilidad de distintos algoritmos MARL ante entornos no estacionarios mediante simulaciones en escenarios sintéticos y en la red vial modelada del microcentro de la Ciudad de Mendoza. Esto permitió establecer una línea base de comparación para, posteriormente, validar empíricamente el desempeño de la arquitectura descentralizada y coordinada propuesta frente a los enfoques de control tradicionales y a metodologías recientes de la literatura.

1.1 Motivación

La motivación principal de este trabajo radica en la marcada asimetría entre una demanda vehicular en constante crecimiento y una infraestructura vial que permanece estática. Dado que ensanchar avenidas o modificar físicamente el trazado urbano en un sector denso como el microcentro de la Ciudad de Mendoza es económica y estructuralmente inviable, la alternativa más sostenible es la optimización lógica de los recursos existentes.

El desarrollo de soluciones basadas en software permite dotar de adaptabilidad a la red semafórica actual. De este modo, un sistema de control adaptativo puede ajustar sus decisiones a las condiciones observadas y buscar una reducción de las demoras sin requerir obras civiles de gran envergadura [MMLK25].

Este trabajo se organiza de la siguiente manera: en el Capítulo 1 se formula el problema de estudio y se exponen las motivaciones que impulsan esta investigación. A continuación, el Capítulo 2 establece las bases teóricas, abordando los conceptos clave de la ingeniería de tránsito y los fundamentos del Aprendizaje por Refuerzo, particularmente en su vertiente multi-agente. Luego, el Capítulo 3 detalla la metodología de trabajo, describiendo tanto los entornos de simulación empleados como la arquitectura algorítmica de los modelos implementados. Posteriormente, en el Capítulo 4 se lleva a cabo un análisis empírico del desempeño de los distintos controladores, evaluándolos en escenarios controlados y en la red urbana simulada del microcentro de la Ciudad de Mendoza. Finalmente, el Capítulo 5 sintetiza los hallazgos principales, destaca las contribuciones metodológicas y sugiere posibles vías para continuar este trabajo en el futuro.

2

Marco Teórico

Este capítulo presenta los conceptos necesarios para plantear el control semafórico como un problema de decisión secuencial. Primero se describen las variables y los elementos de la operación del tránsito; luego se introduce su representación en un simulador microscópico; finalmente se presentan los fundamentos del aprendizaje por refuerzo (RL) y del aprendizaje por refuerzo multiagente (MARL), que permiten formular políticas de control para la red vial.

2.1 Ingeniería de Tránsito y Simulación

La ingeniería de tránsito se define como aquella fase de la ingeniería de transporte —disciplina que se enmarca tradicionalmente dentro de la ingeniería civil— que se ocupa de la planeación segura y eficiente, el proyecto geométrico y la operación del tránsito por calles y carreteras, sus redes, terminales, tierras adyacentes y su relación con otros modos de transporte [CyMR18]. Asimismo, el *Traffic Engineering Handbook* la describe formalmente como la subdisciplina enfocada en la planificación, el diseño geométrico y la operación de redes viales [PW16]. En este trabajo, sus conceptos se abordan desde la perspectiva del modelado computacional y la gestión adaptativa: permiten describir el funcionamiento físico de la red vial, caracterizar la oferta y demanda de circulación, e identificar las variables operacionales sobre las cuales puede intervenir un sistema de control inteligente.

Históricamente, la práctica de la ingeniería de tránsito puso un fuerte énfasis en proveer y ampliar la capacidad física vial mediante la construcción de nuevas obras o el ensanche de calzadas [PW16]. Sin embargo, el crecimiento sostenido del parque automotor evidenció que la ampliación de la infraestructura no constituye una solución definitiva al problema

de la congestión. En áreas urbanas consolidadas, además, modificar el trazado vial resulta costoso y geométricamente inviable [CyMR18]. Por ello, la práctica contemporánea complementa la infraestructura física con estrategias de control dinámico del tránsito, priorizando la optimización de los recursos existentes mediante dispositivos de control semafórico.

La operación de una red vial puede analizarse a partir de la relación entre la demanda vehicular y la capacidad disponible. La demanda vehicular (D) corresponde a la cantidad de vehículos que requieren desplazarse por un determinado sistema vial en un lapso dado, mientras que la oferta vial o capacidad (c) representa la cantidad máxima de vehículos que pueden circular por dicho espacio físico bajo determinadas condiciones de operación [CyMR18]. Cuando la demanda se aproxima a la capacidad, el tránsito se vuelve inestable; cuando la supera ($D > c$), se presentan condiciones de sobresaturación o flujo forzado, caracterizadas por detenciones frecuentes y grandes demoras. En este sentido, la congestión no se entiende solo como una percepción general de mal funcionamiento, sino como un fenómeno con manifestaciones físicas medibles: demoras, colas y detenciones [FA11].

Estas magnitudes cumplen una función doble en el modelo computacional: pueden formar parte de las observaciones y de la señal de recompensa de los agentes, y también se utilizan para evaluar el desempeño del controlador. Su definición permite vincular la descripción física del tránsito con la formulación posterior del problema de aprendizaje.

2.1.1 Terminología de Tránsito

Antes de formular el problema de control, es necesario distinguir algunos términos operativos que se utilizarán a lo largo del capítulo. En particular, se diferencian los movimientos que realizan los vehículos, las fases que habilitan esos movimientos y los intervalos durante los cuales se mantienen las indicaciones semafóricas [Fed08].

En este contexto, se emplearán las siguientes nociones:

- **Intersección (*intersection*), acceso (*approach*) y carril (*lane*):** una intersección es el área física y operacional en la que confluyen dos o más corrientes de tránsito. Cada una de las calzadas de aproximación que ingresan a ella constituye un acceso [PW16], organizándose internamente en uno o más carriles destinados a canalizar los flujos vehiculares [CyMR18].
- **Movimiento vehicular y movimientos en conflicto:** un movimiento vehicular representa la trayectoria específica que describe un vehículo al cruzar la intersección hacia una salida determinada, por ejemplo, un giro o un movimiento directo [W⁺19]. Dos movimientos se consideran en conflicto o incompatibles cuando sus trayectorias se intersectan y generan puntos de conflicto en la intersección (véase la Figura 2.2), lo que impide su servicio simultáneo por razones de seguridad [FA11].

- **Fase (*phase*), ciclo (*cycle*) e intervalo (*interval*):** una fase es una configuración de señales que habilita simultáneamente un conjunto de movimientos compatibles [PW16]. El ciclo corresponde a la secuencia completa de fases necesaria para atender los movimientos de la intersección. Un intervalo es el período durante el cual las indicaciones semafóricas permanecen constantes [CyMR18].

La Figura 2.1 ilustra estos conceptos en una intersección de cuatro accesos. La numeración sigue de manera esquemática una convención habitual de los manuales de temporización semafórica; no representa una configuración obligatoria para el caso de estudio. La figura permite distinguir entre movimiento físico, fase semafórica y usuarios servidos por una misma indicación [Fed08].

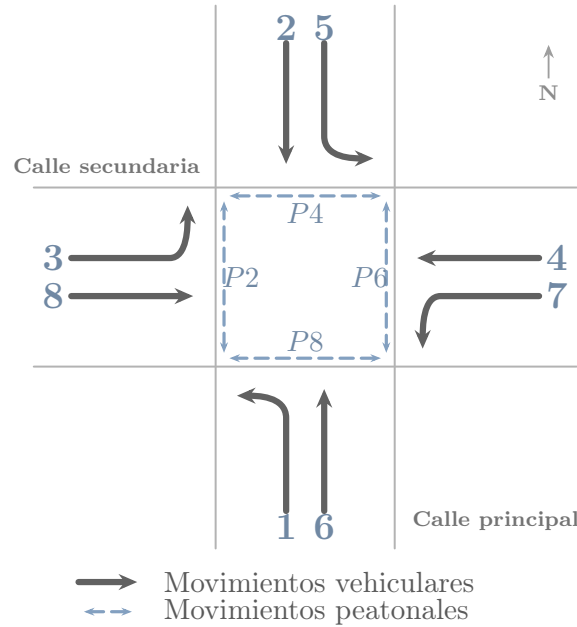


Figura 2.1: Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de FHWA [Fed08].



Figura 2.2: Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.

En arterias urbanas, el tránsito se desarrolla bajo condiciones de flujo interrumpido: los vehículos no avanzan únicamente por su interacción con otros vehículos, sino también por la presencia de intersecciones, señales y dispositivos de control de tránsito [PW16].

Las intersecciones son puntos críticos de la red porque allí confluyen distintos accesos y movimientos vehiculares de cruce, convergencia y divergencia que pueden resultar incompatibles entre sí. Por ello, su operación requiere ordenar estos movimientos y asignar la prioridad de paso de manera segura y eficiente.

El semáforo es uno de los dispositivos de control de tránsito utilizados para regular la circulación en intersecciones. Su función es asignar la prioridad de paso en forma alternada entre corrientes vehiculares incompatibles, evitando que distintos movimientos intenten ocupar simultáneamente el mismo espacio de cruce [FA11]. Desde el punto de vista del modelado algorítmico, esta capacidad de modificar la prioridad de paso convierte al semáforo en el actuador mediante el cual un sistema de control puede intervenir sobre la operación de la intersección y afectar, directa o indirectamente, las demoras y colas observadas en la red.

2.1.2 Fundamentos de la Dinámica Vehicular

Para modelar y optimizar la operación de una red vial, es necesario caracterizar físicamente el movimiento de los vehículos. La ingeniería de tránsito estudia este fenómeno a partir de las características fundamentales de la corriente de tránsito, que permiten describir las condiciones de operación de una vía o intersección mediante magnitudes observables como el flujo, la velocidad y la densidad [PW16]. Cal y Mayor distingue estas magnitudes entre variables principales, que describen el comportamiento agregado del flujo vehicular, y variables asociadas, que expresan relaciones temporales y espaciales entre vehículos consecutivos [CyMR18].

A nivel macroscópico, la corriente de tránsito se representa mediante variables agregadas como la tasa de flujo, la densidad y los promedios de velocidad. A nivel microscópico, se analizan magnitudes propias de vehículos individuales e interacciones entre pares consecutivos, como la velocidad puntual, el intervalo y el espaciamiento [CyMR18]. Esta separación entre escalas es fundamental para el modelado computacional: las variables macroscópicas resumen el estado operacional de los accesos de la red, mientras que las variables microscópicas describen el comportamiento de cada vehículo dentro del simulador [FA11]. La correspondencia y las relaciones de conversión entre las variables de ambas escalas se resumen en la Tabla 2.1.

En el nivel macroscópico, las variables principales de la corriente de tránsito son:

- **Flujo o tasa de flujo (q):** representa la cantidad de vehículos que atraviesan una sección transversal de la vía por unidad de tiempo. Si el período de observación es menor a una hora, se proyecta como una tasa horaria equivalente en vehículos por hora (veh/h), calculada como $q = N/T$, donde N es el número de vehículos observados y T es la duración del intervalo de medición expresado en horas [CyMR18].
- **Densidad o concentración (k):** indica el número de vehículos que ocupan si-

multáneamente una longitud específica de vía, expresada en vehículos por kilómetro (veh/km), calculada como $k = N/d$, donde d representa la longitud del tramo de vía analizado [CyMR18]. En la práctica, se suele estimar indirectamente a través del porcentaje de ocupación temporal registrado por detectores inductivos [PW16].

- **Velocidad media espacial (v_s):** es la velocidad media de los vehículos que ocupan un tramo de vía en un instante determinado. Corresponde al promedio armónico de las velocidades puntuales de los vehículos individuales y representa la magnitud de velocidad consistente con la dinámica del flujo espacial [CyMR18].
- **Velocidad media temporal (v_t):** representa el promedio aritmético de las velocidades de los vehículos que cruzan una sección transversal fija de la vía durante un período de tiempo determinado [CyMR18].

En el nivel microscópico, las variables asociadas describen el movimiento del vehículo individual y la separación entre vehículos consecutivos (véase la Figura 2.3):

- **Velocidad puntual (v_i):** es la velocidad instantánea a la que un vehículo individual (i) atraviesa una sección transversal específica de la vía.
- **Intervalo temporal (h):** es el tiempo transcurrido entre el paso de dos vehículos consecutivos por una sección fija de la calzada, medido respecto a puntos homólogos, por ejemplo, los parachoques delanteros [CyMR18]. El intervalo promedio en segundos (h_{prom}) se relaciona con la tasa de flujo como $h_{\text{prom}} = 3600/q$.
- **Espaciamiento simple (s):** es la distancia física entre puntos homólogos de dos vehículos consecutivos en un instante de tiempo [CyMR18]. El espaciamiento promedio en metros (s_{prom}) se relaciona de forma inversa con la densidad de la corriente de tránsito como $s_{\text{prom}} = 1000/k$.

Tabla 2.1: Comparación y correspondencia de variables de tránsito en escalas macroscópica y microscópica.

Escala	Variable Principal	Relación / Conversión
Macroscópica	Flujo (q , veh/h)	$q = 3600/h_{\text{prom}}$
	Densidad (k , veh/km)	$k = 1000/s_{\text{prom}}$
	Velocidad media espacial (v_s , km/h)	$q = k \cdot v_s$
	Velocidad media temporal (v_t , km/h)	$v_t = v_s + \frac{\sigma_s^2}{v_s}$ (Relación de Wardrop)
Microscópica	Velocidad puntual (v_i , km/h)	Variable individual de cada vehículo
	Intervalo temporal (h , s)	Tiempo entre pasos de vehículos sucesivos
	Espaciamento simple (s , m)	Distancia espacial entre parachoques homólogos

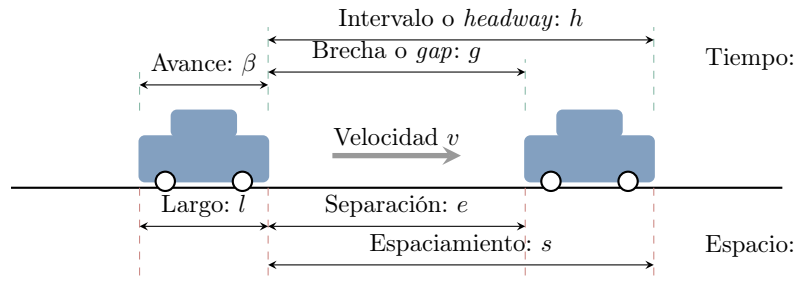


Figura 2.3: Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamento (s).

Estas variables macroscópicas se relacionan de manera directa a través de la ecuación fundamental del flujo de tránsito:

$$q = k \cdot v_s \quad (2.1)$$

donde q es la tasa de flujo, k es la densidad y v_s es la velocidad media espacial. Esta ecuación utiliza la velocidad media espacial, ya que relaciona el flujo y la cantidad de vehículos presentes en un tramo. La velocidad media temporal (v_t) difiere de ella porque asigna mayor peso a los vehículos más rápidos que cruzan un punto de medición durante un intervalo dado. La relación entre ambas variables se expresa mediante la relación de Wardrop [FA11]:

$$v_t = v_s + \frac{\sigma_s^2}{v_s} \quad (2.2)$$

donde σ_s^2 representa la varianza de la distribución de las velocidades de los vehículos en el espacio. Dado que la varianza es no negativa ($\sigma_s^2 \geq 0$), se cumple que $v_t \geq v_s$, con igualdad

únicamente cuando todos los vehículos circulan a la misma velocidad. En una intersección semaforizada, una indicación roja puede reducir localmente la velocidad media y aumentar la acumulación de vehículos en los accesos. Estos efectos se manifiestan en la formación de colas y en el incremento de las demoras [CyMR18].

Métricas de Ineficiencia Operativa

La interrupción del flujo causada por semáforos o saturación genera efectos adversos que los sistemas de control buscan mitigar. Siguiendo la terminología de la ingeniería de tránsito, las principales manifestaciones medibles de esta ineficiencia son las demoras, las colas y las detenciones [FA11]:

- **Demoras:** Representan el tiempo adicional que un vehículo debe invertir para atravesar una intersección respecto del tiempo que le tomaría circular a velocidad de flujo libre. En intersecciones semaforizadas, una parte de esa demora aparece por la espera normal durante la luz roja. Otras demoras se producen cuando las llegadas de vehículos varían, cuando la demanda se acerca o supera la capacidad de la intersección, o cuando quedan colas sin disiparse al finalizar el verde, de modo que algunos vehículos deben esperar más de un ciclo para cruzar.
- **Colas:** Corresponden al número de vehículos acumulados detrás de una línea de detención a la espera de ser servidos. Esta medida espacial es especialmente relevante porque, cuando la cola supera la longitud del arco de aproximación, puede bloquear la intersección aguas arriba y propagar la congestión al resto de la red.
- **Detenciones:** Cuantifican la cantidad de veces que un vehículo debe reducir su velocidad hasta cero durante el recorrido. Una alta frecuencia de detenciones deteriora la continuidad operativa del flujo y suele asociarse con mayores procesos de aceleración y desaceleración.

Estas magnitudes permiten describir el desempeño operacional de una intersección y de la red. En la formulación computacional, también pueden utilizarse para construir observaciones, señales de recompensa o criterios de evaluación del controlador.

Descripción Probabilística del Flujo y Pelotones Vehiculares

En el modelado de tránsito, las llegadas vehiculares no siempre pueden representarse adecuadamente como una secuencia determinista y constante. Para flujos vehiculares bajos y medios, y cuando las llegadas no están fuertemente condicionadas por semáforos previos, es habitual aproximar el número de vehículos que llega a una sección durante un intervalo de tiempo mediante un proceso de Poisson [CyMR18].

Esta aproximación supone independencia estadística entre llegadas y permite describir la variabilidad observada en la demanda de acceso a una intersección [CyMR18]. La probabilidad discreta $P(X)$ de que lleguen exactamente X vehículos en un intervalo de tiempo

se define como:

$$P(X) = \frac{m^X \cdot e^{-m}}{X!} \quad (2.3)$$

donde $X \in \mathbb{N}_0$ es la variable aleatoria discreta que representa el número de arribos vehiculares, $m > 0$ representa el valor esperado o media de llegadas en dicho intervalo, y e es la base del logaritmo natural. Bajo el mismo supuesto, los intervalos de tiempo continuos (h) medidos entre vehículos sucesivos se describen mediante una distribución de probabilidad exponencial negativa [CyMR18]:

$$P(h \geq t) = e^{-\frac{q}{3600} \cdot t} \quad (2.4)$$

donde h es el intervalo temporal en segundos, q es el flujo en veh/h, y t es el umbral de tiempo analizado.

Esta representación probabilística asume independencia entre arribos sucesivos, siendo una aproximación válida únicamente para flujos vehiculares bajos o moderados bajo condiciones de flujo libre, donde no exista la influencia de semáforos cercanos aguas arriba [CyMR18]. Sin embargo, este supuesto de independencia decae en condiciones de congestión o en redes viales urbanas semaforizadas. En estos casos, la operación de los semáforos interrumpe periódicamente el flujo y agrupa a los vehículos en pelotones (*platoons*), induciendo una fuerte dependencia espacio-temporal y correlación entre arribos consecutivos. La Figura 2.4 presenta una comparación conceptual de la distribución resultante de los intervalos temporales entre vehículos bajo estas dos condiciones operativas.

Esta limitación de los modelos analíticos tradicionales justifica el uso de simulación microscópica para estudiar el control semafórico bajo condiciones variables. En SUMO, la formación, dispersión e interacción de los pelotones emergen de la dinámica simulada de los vehículos y de sus cambios de carril. El uso de distintos perfiles de demanda y semillas de simulación permite evaluar el comportamiento del controlador frente a variaciones en las condiciones de operación.

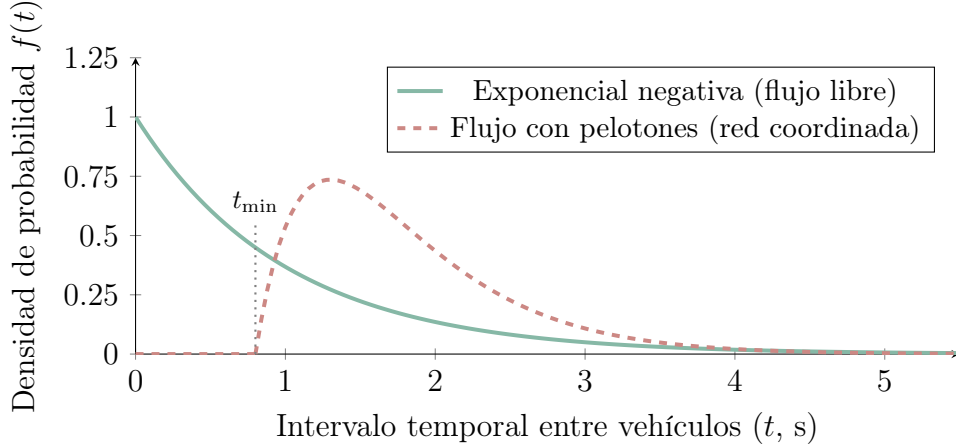


Figura 2.4: Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de pelotones. Las curvas representan situaciones conceptuales y no una distribución empírica particular.

2.1.3 Sistemas de Control de Tránsito

Para ordenar los movimientos en conflicto y regular el paso en las intersecciones urbanas se utilizan sistemas de control de tránsito [PW16]. El semáforo asigna el derecho de paso de manera alternada mediante indicaciones luminosas rojas, amarillas y verdes [CyMR18].

En el modelo computacional, el semáforo actúa como el mecanismo mediante el cual el controlador interviene sobre la red. La selección de fases, la duración del verde efectivo y la coordinación entre intersecciones modifican la prioridad de paso y afectan la evolución de las colas y las demoras.

Sobre esta base terminológica, la programación temporal de un semáforo se describe mediante la duración de las fases, la longitud de ciclo y la definición de los intervalos de transición y despeje. Estos parámetros determinan cuánto tiempo recibe prioridad cada conjunto de movimientos compatibles y condicionan directamente la capacidad de descarga, las demoras y la seguridad operacional de la intersección [PW16].

En relación con los giros izquierdos, los manuales de temporización distinguen entre operación permisiva, protegida y protegida-permisiva. En una fase permisiva, el giro se ejecuta aprovechando brechas en la corriente opuesta; en una fase protegida, el giro recibe una indicación exclusiva y los movimientos conflictivos permanecen detenidos; en una fase protegida-permisiva, ambas condiciones se combinan durante distintos intervalos del ciclo [Fed08]. Para el presente trabajo, la distinción resulta útil porque muestra que una fase no es solo un color del semáforo, sino una regla operacional que habilita ciertos movimientos, impone cesión de paso en otros y restringe los movimientos en conflicto.

Para garantizar la seguridad vial, las transiciones entre fases no pueden ejecutarse de manera instantánea, sino que deben respetar intervalos de cambio y despeje ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) [PW16]. En esta investigación, esos tiempos se consideran restricciones operativas de seguridad previamente definidas en la configuración del simulador y del sistema

semafórico. En consecuencia, su determinación no forma parte del proceso de decisión del agente, cuyo control se limita a la selección o extensión de fases dentro de un esquema temporal compatible con dichas restricciones.

Modelado de la Capacidad y el Verde Efectivo

Desde el punto de vista operativo, la capacidad de descarga de un acceso semaforizado depende del tiempo efectivo de servicio que recibe cada fase y de la tasa con la que la fila puede disiparse cuando el movimiento tiene prioridad [FA11]. En la práctica, la descarga observada no depende solo de la duración nominal del verde, sino también de pérdidas de arranque, transiciones y otras restricciones temporales del control, como se ilustra conceptualmente en la Figura 2.5. En este trabajo, estos efectos no se modelan mediante fórmulas analíticas de capacidad, sino mediante la dinámica microscópica del simulador y la configuración temporal del sistema semafórico, ya que esas interacciones determinan las colas, demoras y detenciones que el controlador busca mejorar.

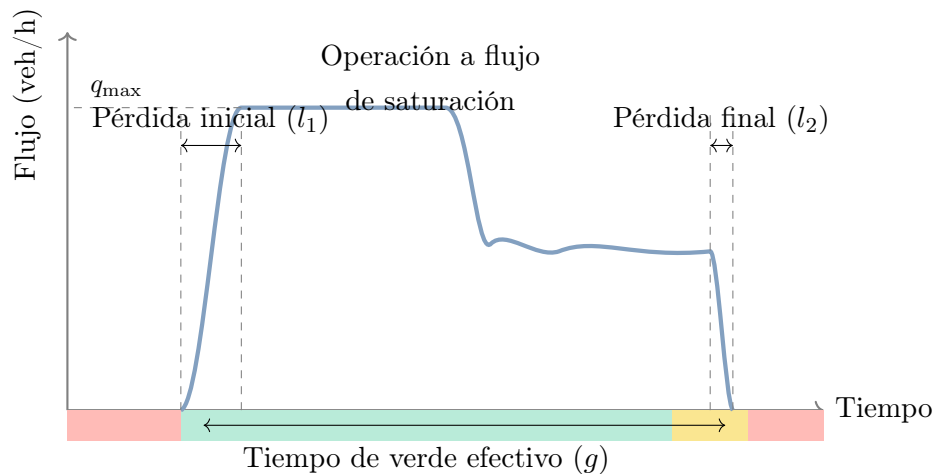


Figura 2.5: Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.

Clasificación de los Controladores de Semáforos

De acuerdo con su mecanismo operativo y su grado de respuesta a la demanda vehicular, los controladores de semáforos se clasifican clásicamente en control de tiempo fijo y control accionado por el tránsito [PW16]. En el primero, los planes de señales se repiten de manera preprogramada a partir de datos históricos y no cambian en respuesta a la demanda instantánea. En el segundo, la intersección utiliza detectores en los accesos para variar dinámicamente el inicio o la duración de las fases según la demanda observada en tiempo real.

Incluso en los esquemas accionados, la operación permanece sujeta a parámetros límite de seguridad y funcionamiento, como el verde mínimo ($g_{\min}$) y el verde máximo ($g_{\max}$) [PW16]. Estos parámetros restringen cuánto puede sostenerse o finalizarse una fase y, en

este trabajo, forman parte de las condiciones operativas dentro de las cuales el controlador toma decisiones.

2.1.4 Simulación de Tránsito

Debido a la densidad y complejidad de la red urbana moderna, la sincronización y optimización de los tiempos de semáforos no pueden resolverse adecuadamente con modelos estáticos bajo condiciones dinámicas, por lo que se recurre a aproximaciones computacionales basadas en simulación de tránsito [PW16]. De acuerdo con la taxonomía clásica [FA11], estos modelos pueden organizarse en tres niveles de resolución. Los modelos macroscópicos describen el tránsito de manera agregada mediante variables como flujo, densidad y velocidad; los modelos mesoscópicos representan entidades intermedias, como grupos o pelotones de vehículos. Ambos niveles son útiles para estudiar fenómenos de red, pero no ofrecen el detalle necesario para observar cómo las decisiones semaforicas afectan, paso a paso, la formación de colas y la interacción local entre vehículos en una intersección.

Para el presente trabajo, el nivel de mayor interés es el microscópico, ya que representa explícitamente a cada vehículo, su ruta y su movimiento dentro de la red [FA11]. SUMO (*Simulation of Urban MObility*) se ubica en esta categoría: es un simulador de tránsito microscópico, multimodal y de código abierto, diseñado para representar demandas vehiculares sobre redes viales y evaluar problemas de gestión del tránsito [ALBBW⁺18]. En SUMO, la evolución vehicular es continua en el espacio y discreta en el tiempo; además, el entorno permite modelar calles con múltiples carriles, cambios de carril, distintos tipos de vehículos, rutas individuales, semáforos y salidas de simulación a nivel de vehículo, carril, arco o detector [DLR25]. Esta granularidad vuelve a la simulación microscópica especialmente adecuada para estudiar control semaforico mediante aprendizaje por refuerzo. Las decisiones sobre fases y tiempos de servicio no se evalúan mediante una fórmula cerrada de demora, sino observando sus consecuencias sobre la dinámica simulada: colas, tiempos de espera, detenciones, ocupación de carriles y propagación de congestión. Por ello, la simulación microscópica constituye el puente natural entre el problema de tránsito y su formulación computacional, al permitir que el controlador interactúe con un entorno dinámico y medible durante la ejecución.

Modelos Microscópicos de Comportamiento Vehicular

En este contexto, el simulador de tránsito constituye el entorno operativo sobre el cual se formaliza el problema de control semaforico. En SUMO, una intersección semaforizada se estructura mediante tres componentes geométricos y lógicos principales bajo el sentido de circulación por la derecha (véase la Figura 2.6) [DLR25]:

- **Carriles entrantes (L_{in}) y salientes (L_{out}):** Los carriles entrantes conducen el flujo vehicular hacia la línea de detención de la intersección por la derecha de cada

acceso, mientras que los carriles salientes reciben el flujo descargado tras cruzar el nodo.

- **Líneas de detención (*stop lines*):** Delimitación física continua en color blanco en el límite de los carriles entrantes L_{in} con el área interna de cruzamiento (*junction*).
- **Enlaces de conexión (*links*):** Rutas internas que conectan un carril entrante específico con un carril saliente de destino. A cada enlace se le asigna un estado semafórico instantáneo (G para verde prioritario, g para verde permisivo, y r para rojo o detención). Como se ilustra en la Figura 2.6, mientras los movimientos directos se encuentran habilitados en verde (G), el giro a la izquierda en conflicto (representado por el enlace en rojo salmón) se mantiene retenido en luz roja (r) para prevenir colisiones.

Esta representación es compatible con la formulación algorítmica del problema, donde una acción consiste en seleccionar una fase que habilita un conjunto de enlaces compatibles mientras mantiene en rojo (r) los movimientos en conflicto.

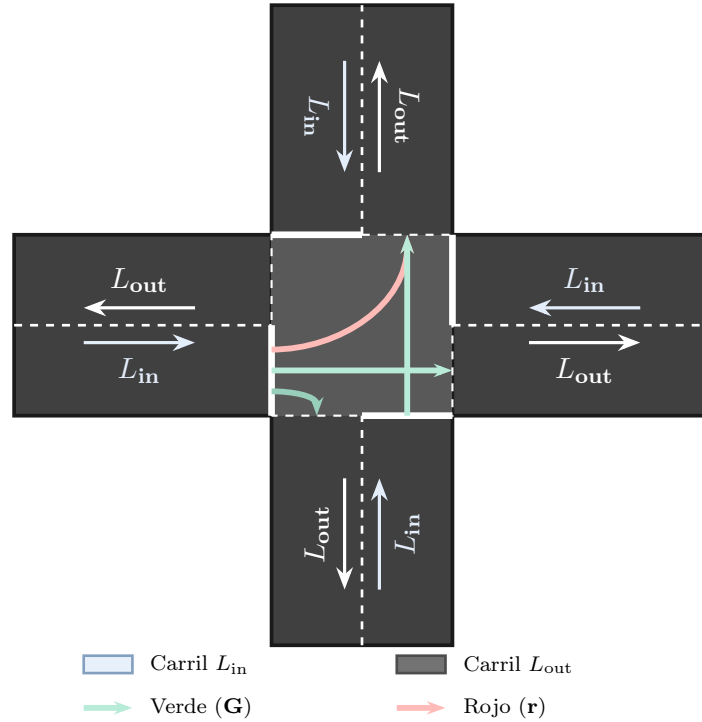


Figura 2.6: Elementos geométricos y conexiones de una intersección semaforizada en SUMO bajo sentido de circulación por la derecha (Argentina): carriles entrantes (L_{in}), salientes (L_{out}), líneas de detención y enlaces de cruzamiento (*links*) en verde prioritario (G) y rojo retenido en conflicto (r).

Componentes y Configuración de SUMO

Para la generación, edición y control en tiempo real de los escenarios de simulación microscópica, el entorno SUMO se complementa con un conjunto integrado de herramientas

y librerías auxiliares [ALBBW⁺18]:

- ***netedit***: Herramienta gráfica que permite proyectar y editar la geometría y la lógica operacional de la red vial de forma interactiva. Facilita la creación y modificación bidireccional de nodos, accesos, carriles, restricciones de giro, conexiones internas (*links*), sensores inductivos y programas de temporización semafórica.
- ***osm Web Wizard***: Herramienta automatizada con interfaz web que permite importar áreas geográficas reales desde OpenStreetMap (OSM) y transformarlas al formato nativo de SUMO, extrayendo información de la red como la topología urbana y los límites de velocidad.
- ***TraCI (Traffic Control Interface)***: Protocolo de comunicación cliente-servidor mediante sockets TCP que habilita la interacción dinámica a paso discreto entre la simulación y scripts externos de control [DLR25]. A través de TraCI, los agentes de aprendizaje por refuerzo recolectan observaciones del estado del tránsito (colas, velocidades, ocupación) e imponen decisiones sobre las fases del semáforo.

Desde la perspectiva del modelado computacional, la ejecución de una simulación microscópica en SUMO se organiza mediante archivos de configuración en formato XML [DLR25]. En los escenarios considerados, sus componentes principales se resumen en la Tabla 2.2:

- **Archivo de red (.net.xml)**: Define la estructura física e infraestructura de la red vial, incluyendo las coordenadas nodales, la geometría de arcos, los carriles por acceso, las conexiones de cruzamiento y la lógica de accionamiento semafórico.
- **Archivo de rutas (.rou.xml)**: Especifica las características de la flota vehicular (dimensiones, aceleración, velocidad deseada y modelo de seguimiento) junto con los itinerarios o flujos de origen a destino.
- **Archivo TAZ (.taz.xml, *Traffic Analysis Zones*)**: Delimita las zonas de análisis de tráfico para asignar espacialmente las matrices de origen y destino de los desplazamientos.
- **Archivo de elementos adicionales (.add.xml)**: Incorpora dispositivos de medición (como detectores de bucle inductivo E1 y E2), paradas de transporte público o programas semafóricos personalizados.
- **Archivo maestro de configuración (.sumocfg)**: Integra las referencias a los archivos de red, rutas y adicionales, y fija los parámetros del horizonte temporal de simulación (duración del paso Δt , tiempos de inicio y fin, e indicadores de salida).

Tabla 2.2: Archivos de configuración empleados en los escenarios de simulación en SUMO [DLR25].

Archivo XML	Extensión	Función en el Modelo Computacional
Red Vial	.net.xml	Topología física, accesos, carriles, enlaces y semáforos.
Rutas y Demanda	.rou.xml	Tipos de vehículos, modelos de seguimiento e itinerarios.
Zonas TAZ	.taz.xml	Zonas de origen y destino para matrices de demanda.
Elementos Adicionales	.add.xml	Sensores inductivos E1/E2, telemetría y programas auxiliares.
Configuración Maestra	.sumocfg	Unificación de archivos, resolución temporal (Δt) y registros.

Métricas de Evaluación Operacional y Aprendizaje

Para evaluar rigurosamente el impacto de las estrategias de control semafórico tanto desde la perspectiva de la ingeniería de tránsito como del aprendizaje automático, este trabajo distingue cuatro tipos de indicadores para la evaluación y el diagnóstico del entrenamiento:

- **Emisión de gases CO₂:** Cuantifica el impacto ambiental y la ineficiencia energética del flujo vehicular. En SUMO, las emisiones se estiman mediante modelos microscópicos como HBEFA o PHEMlight [DLR25], relacionando la masa de CO₂ emitida en gramos con los perfiles de velocidad, aceleración y tiempo en ralentí (*idling*) de cada vehículo. Una reducción de la masa total de CO₂ puede asociarse con una operación más fluida y una menor frecuencia de detenciones.
- **Tiempo de Espera Promedio Global y por Agente:** Métrica operacional primaria en ingeniería de tránsito [CyMR18]. El tiempo de espera cuantifica la acumulación en segundos que un vehículo permanece detenido (velocidad $v < 0.1$ m/s) antes de cruzar la intersección. Se evalúa a nivel local por cada agente i (intersección) y de forma agregada a nivel global de la red vial, permitiendo medir la reducción directa de la demora operacional.
- **Recompensa Acumulada Descontada:** Métrica interna de RL/MARL definida como el retorno episódico $G_0 = \sum_{t=0}^T \gamma^t R_t$. Expresa la capacidad del agente para maximizar de forma persistente su objetivo escalar proyectado en el tiempo [SB18].
- **Valores de pérdida (*loss values*):** Indicadores de diagnóstico durante el entrenamiento en DRL/MADRL [YVV⁺22]. Incluyen la pérdida de política o actor ($\mathcal{L}^{\text{CLIP}}$), la pérdida de error cuadrático medio de la función de valor del crítico ($\mathcal{L}^{\text{VF}}$)

y el término de entropía de la política ($S[\pi]$). Su seguimiento permite identificar inestabilidades del gradiente y cambios en el proceso de optimización; por sí solo, no demuestra la calidad operacional del controlador.

En este marco, SUMO no solo permite reproducir la evolución microscópica de la red, sino también interactuar con ella mediante TraCI. La posibilidad de observar el estado del tránsito y modificar las fases semafóricas en pasos discretos convierte el control semafórico en un problema de decisión secuencial, lo que motiva la introducción del aprendizaje por refuerzo en la siguiente sección [DLR25].

2.2 Aprendizaje por Refuerzo (RL)

El aprendizaje automático (*Machine Learning*, ML) suele dividirse en tres paradigmas. En el aprendizaje supervisado (*Supervised Learning*, SL), el modelo se entrena con ejemplos etiquetados que indican la salida esperada. En el aprendizaje no supervisado (*Unsupervised Learning*, UL), se busca descubrir estructura en datos no etiquetados, por ejemplo mediante agrupamiento o reducción de dimensionalidad. El aprendizaje por refuerzo (*Reinforcement Learning*, RL) aborda un problema distinto: una entidad aprende mediante interacción con un entorno qué acciones elegir para maximizar una recompensa acumulada [SB18].

Esta diferencia es central. En RL no existe, en general, un supervisor que indique cuál era la acción correcta en cada situación. El agente aprende por experiencia, ensayando acciones, observando sus consecuencias y ajustando su comportamiento a partir de las recompensas obtenidas. Además, las consecuencias relevantes de una acción no siempre se manifiestan de forma inmediata: una decisión puede producir una recompensa baja en el corto plazo, pero conducir a estados favorables en el futuro. Por este motivo, el aprendizaje por refuerzo se ocupa explícitamente del problema de la recompensa diferida y de la asignación de crédito a decisiones pasadas [SB18].

La formulación básica distingue entre el *agente* (*agent*), que aprende y toma decisiones, y el *entorno* (*environment*), que representa todo aquello externo al agente con lo cual este interactúa. En cada paso temporal, el agente observa una situación del entorno, selecciona una acción y recibe como respuesta una nueva situación junto con una recompensa. El comportamiento del agente está determinado por una *política* (*policy*), denotada por π , que especifica cómo se eligen las acciones a partir de los estados u observaciones disponibles. La *señal de recompensa* (*reward signal*) define el objetivo inmediato del problema, mientras que la *función de valor* estima la conveniencia de estados o acciones en términos de recompensa futura esperada. Sutton y Barto también distinguen el *modelo* del entorno, entendido como una descripción que permite predecir sus transiciones y recompensas; cuando dicho modelo no se conoce o no se utiliza explícitamente, se habla de métodos libres de modelo (*model-free*).

En el control semafórico, el controlador de una intersección puede interpretarse como el agente y la red vial simulada en SUMO como el entorno. La selección de una fase constituye la acción, mientras que la evolución posterior del tránsito produce nuevas observaciones y una señal de recompensa que orienta el aprendizaje [DLR25].

Un aspecto característico de RL es el dilema entre exploración y explotación. El agente debe explotar acciones que, según su experiencia, producen buenos resultados, pero también debe explorar alternativas menos conocidas que podrían conducir a políticas superiores. Si explora demasiado, desperdicia oportunidades de obtener recompensa; si explota demasiado pronto, puede converger a comportamientos subóptimos. Este equilibrio es especialmente relevante en problemas de control, donde las acciones modifican la experiencia futura disponible para el aprendizaje [SB18].

2.2.1 El Proceso de Decisión de Markov (MDP)

El marco matemático estándar para estudiar problemas de aprendizaje por refuerzo de un solo agente es el Proceso de Decisión de Markov (*Markov Decision Process*, MDP). En un MDP finito, el agente y el entorno interactúan en una secuencia de pasos temporales discretos $t = 0, 1, 2, \dots$. En cada instante t , el agente recibe una representación del estado del entorno $S_t \in \mathcal{S}$ y selecciona una acción $A_t \in \mathcal{A}(S_t)$. Como consecuencia de esa acción, el entorno emite una recompensa numérica $R_{t+1} \in \mathcal{R}$ y transita a un nuevo estado S_{t+1} [SB18, ACS24]. Este ciclo se representa en la Figura 2.7.

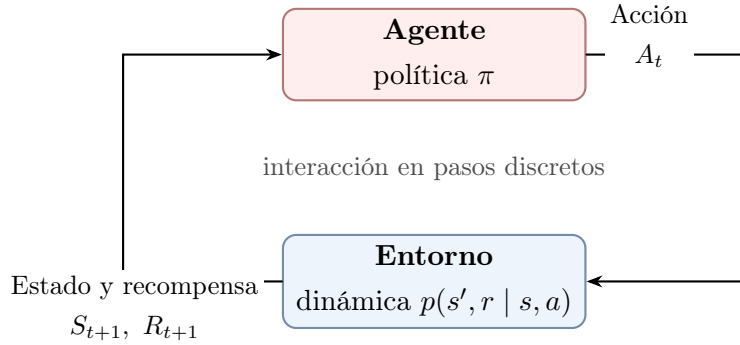


Figura 2.7: Interacción agente-entorno en un proceso de decisión de Markov [SB18].

De forma compacta, un MDP finito queda caracterizado por la tupla $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, P la dinámica de transición probabilística, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento.

La dinámica del entorno queda definida por la función $p(s', r | s, a)$, que especifica la probabilidad condicional conjunta de observar el próximo estado s' y la recompensa r al ejecutar la acción a en el estado s :

$$p(s', r | s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a\} \quad (2.5)$$

donde $S_t \in \mathcal{S}$ representa el estado en el instante t , $A_t \in \mathcal{A}$ es la acción seleccionada,

$R_{t+1} \in \mathbb{R}$ es la recompensa escalar recibida y $S_{t+1} \in \mathcal{S}$ es el nuevo estado resultante.

La propiedad de Markov establece que, dado el estado y la acción actuales, la distribución del próximo estado y recompensa no depende explícitamente de la historia completa de estados y acciones anteriores. En otras palabras, el estado debe contener toda la información relevante del pasado para predecir la evolución futura del proceso [SB18].

Una suposición habitual del MDP es que el agente observa un estado suficientemente informativo del entorno. Sin embargo, en muchas aplicaciones reales el agente solo recibe observaciones parciales o ruidosas. En estos casos, el marco se generaliza a un Proceso de Decisión de Markov Parcialmente Observable (*Partially Observable Markov Decision Process*, POMDP), donde el historial de observaciones puede ser necesario para inferir el estado subyacente [ACS24].

Para formalizar la meta de maximizar la recompensa a largo plazo se define el retorno descontado G_t , que incorpora un factor $\gamma \in [0, 1)$ para ponderar la importancia de las recompensas futuras [SB18]:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.6)$$

donde G_t representa la suma acumulada descontada de recompensas futuras a partir del instante t , y γ es el factor de descuento. Un valor de γ cercano a cero prioriza recompensas inmediatas, mientras que un valor cercano a uno otorga un peso significativo a recompensas futuras de largo plazo [ACS24].

2.2.2 Métodos Basados en Valor y Basados en Política

Los algoritmos de RL pueden agruparse, a grandes rasgos, en métodos basados en valor y métodos basados en política. Esta distinción no agota la taxonomía del campo, pero permite introducir las familias de algoritmos más relevantes para este trabajo.

En los *métodos basados en valor*, la estrategia consiste en estimar qué tan conveniente es encontrarse en un estado o ejecutar una acción determinada. Esto se formaliza mediante la *función de valor de estado* $v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s]$ y la *función de valor de acción* $q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$. Ambas funciones satisfacen ecuaciones recursivas conocidas como *ecuaciones de Bellman*, que expresan la relación entre el valor de una situación actual y el valor esperado de sus posibles sucesores [SB18].

Para una política fija, las ecuaciones de Bellman permiten evaluar el valor esperado de los estados y acciones. Cuando el objetivo es encontrar la mejor acción disponible, se utilizan las ecuaciones de optimalidad de Bellman:

$$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right]. \quad (2.7)$$

Esta relación justifica el objetivo de actualización de Q-learning y, posteriormente, de

DQN. La maximización sobre las acciones futuras convierte el problema de evaluación de una política en un problema de control orientado a aproximar la función de acción-valor óptima q_* [SB18].

Cuando la dinámica del entorno no se conoce completamente, estas funciones pueden estimarse a partir de la experiencia. Los métodos de Diferencia Temporal (*Temporal-Difference*, TD) combinan el muestreo de transiciones reales con actualizaciones basadas en estimaciones previas, mecanismo conocido como *bootstrapping*. Un ejemplo central es *Q-learning*, que actualiza la función de acción-valor hacia una estimación de la función óptima $q_*(s, a)$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \quad (2.8)$$

donde $Q(S_t, A_t)$ es la estimación del valor de la acción A_t en el estado S_t , $\alpha \in (0, 1]$ representa la tasa de aprendizaje (*learning rate*), R_{t+1} es la recompensa observada, γ es el factor de descuento, y $\max_a Q(S_{t+1}, a)$ estima el valor máximo alcanzable en el siguiente estado S_{t+1} .

Q-learning es un método *off-policy*: la política que genera las transiciones puede diferir de la política cuyo valor óptimo se estima. En cambio, los métodos de gradiente de política utilizan, en principio, muestras generadas por la política que se está optimizando y se consideran *on-policy*. Esta distinción resulta central para comparar IDQN con IPPO, MAPPO y HAPPO, especialmente en entornos multiagente donde las políticas de los demás agentes cambian durante el entrenamiento.

Por el contrario, los *métodos basados en política* parametrizan directamente la política $\pi(a | s, \theta) = \Pr\{A_t = a | S_t = s, \theta_t = \theta\}$, donde $\theta \in \mathbb{R}^d$ representa el vector de parámetros ajustables (pesos neuronales). En lugar de derivar la acción exclusivamente de una función de valor, ajustan los parámetros θ para maximizar una medida de rendimiento escalar $J(\theta)$. Para ello aplican ascenso de gradiente sobre el rendimiento esperado:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t) \quad (2.9)$$

donde $\nabla J(\theta_t)$ representa el gradiente de la función de rendimiento respecto a los parámetros de la política. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) proporciona la base teórica para estimar este gradiente sin derivar explícitamente la distribución de estados inducida por la política [SB18].

Los métodos actor-crítico combinan una política parametrizada con una estimación de valor y actualizan ambos componentes de manera concurrente [KT03, SB18]. El **actor** representa la política $\pi_\theta(a | s)$ y determina la distribución utilizada para seleccionar la acción. El **crítico** no controla el entorno: estima, por ejemplo, el valor de estado $V_\phi(s)$ y produce una señal que indica si el resultado observado fue mejor o peor que lo esperado bajo la política vigente. Los parámetros θ y ϕ pueden pertenecer a redes separadas o a

una arquitectura con capas compartidas y cabezales distintos.

En una formulación actor-crítico de un paso, después de ejecutar A_t en S_t , el entorno devuelve la recompensa R_{t+1} , el estado S_{t+1} y el indicador terminal d_{t+1} . El crítico calcula el error de diferencia temporal

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t), \quad (2.10)$$

donde el factor $(1 - d_{t+1})$ elimina el valor del estado siguiente cuando finaliza el episodio. Si V_ϕ aproxima correctamente a V^π , δ_t constituye una estimación de un paso de la ventaja de la acción ejecutada. Por ello, un valor positivo indica un resultado superior al esperado para ese estado y uno negativo indica lo contrario. El crítico ajusta ϕ para reducir el error entre $V_\phi(S_t)$ y el objetivo temporal $R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1})$, mientras que el actor utiliza la misma señal para reforzar o desalentar la acción observada:

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t). \quad (2.11)$$

Esta forma canónica permite explicar el intercambio de información entre ambos componentes, aunque los algoritmos modernos suelen sustituir δ_t por estimadores de ventaja que combinan varios pasos, como GAE, y realizan las actualizaciones sobre mini-lotes.

La Figura 2.8 resume este circuito para el control semafórico. La observación del tránsito alimenta al actor y al crítico; el actor selecciona una fase, el entorno simulado produce la siguiente observación y la recompensa, y el crítico transforma esa transición en una señal de evaluación. Esta señal guía el aprendizaje, pero no se utiliza como una orden directa sobre el entorno.

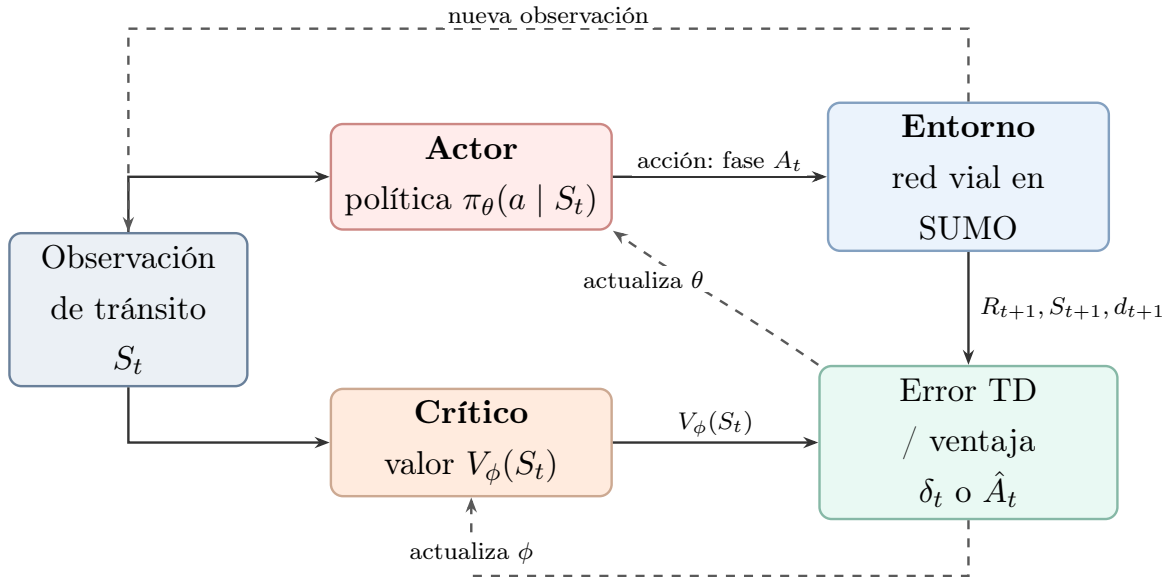


Figura 2.8: Arquitectura actor-crítico aplicada al control semafórico. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].

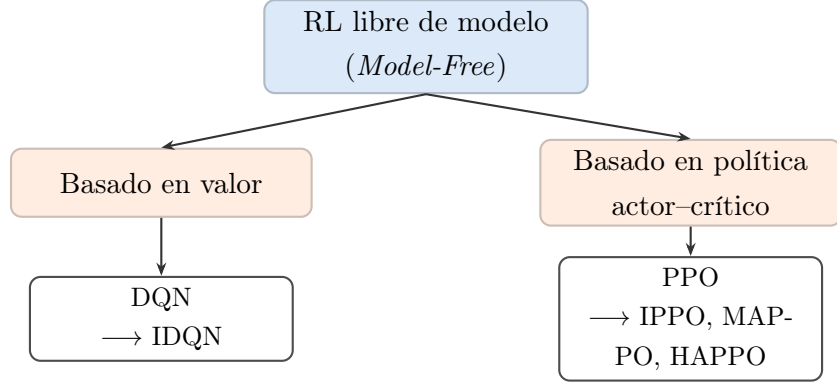


Figura 2.9: Relación entre las familias de algoritmos utilizadas en esta investigación. Los cuatro métodos evaluados son libres de modelo y se derivan de las líneas DQN y PPO.

La Figura 2.9 ubica los algoritmos evaluados dentro de las familias de métodos libres de modelo basados en valor y basados en política.

2.2.3 Aprendizaje Profundo y Redes Neuronales

El aprendizaje profundo (*Deep Learning*) es un subcampo del aprendizaje automático centrado en el entrenamiento de redes neuronales con múltiples capas de parámetros ajustables. Estas arquitecturas permiten aprender representaciones jerárquicas y no lineales a partir de los datos, en lugar de depender exclusivamente de características fijadas de antemano [BB24, GBC16]. En este trabajo, esta capacidad se utiliza para aproximar funciones de valor y políticas de control cuando una representación tabular no resulta adecuada.

La Unidad Neuronal y Redes *Feedforward*

En problemas de control complejos, los métodos tabulares se vuelven impracticables debido a las restricciones de memoria y cómputo asociadas con espacios de estados grandes o continuos [SB18]. La aproximación de funciones aborda esta limitación mediante una representación parametrizada, de modo que la información aprendida en una situación pueda generalizarse a otras situaciones con características similares. Las redes neuronales constituyen una familia de aproximadores de funciones que puede utilizarse para este propósito [BB24].

El componente básico de una red neuronal es la unidad neuronal individual. Dada una entrada $\mathbf{x}$, la unidad asigna un peso a cada componente de esa entrada, suma un sesgo y transforma el resultado mediante una función de activación. La suma ponderada más el sesgo se denomina preactivación, mientras que la salida de la función de activación constituye la respuesta de la unidad [GBC16].

$$a_{k,u} = \mathbf{w}_{k,u}^\top \mathbf{x}_{k-1} + b_{k,u}, \quad h_{k,u} = g_k(a_{k,u}) \quad (2.12)$$

En la ecuación 2.12, $\mathbf{x}_{k-1}$ es el vector de entrada a la capa k , $\mathbf{w}_{k,u}$ contiene los pesos

asociados con la unidad u , $b_{k,u}$ es el sesgo y g_k es la función de activación. La función de activación introduce la no linealidad necesaria: la composición de transformaciones exclusivamente lineales podría reducirse a una única transformación lineal, aun cuando se utilizaran varias capas [GBC16].

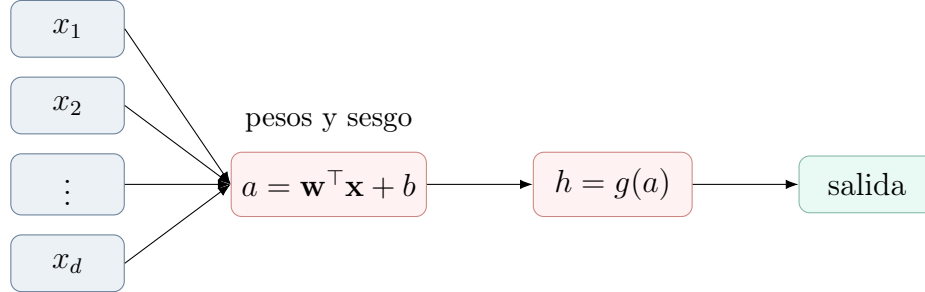


Figura 2.10: Operaciones de una unidad neuronal artificial: combinación afín de las entradas y aplicación de una función de activación.

Las funciones de activación más habituales en las capas ocultas consideradas aquí son [GBC16]:

- **ReLU:** $g(z) = \max\{0, z\}$. Es sencilla de optimizar y conserva una derivada no nula en la región activa, aunque no es diferenciable exactamente en $z = 0$; en la práctica se utiliza una derivada lateral o subgradiente.
- **Leaky ReLU:** $g(z) = \max\{0, z\} + \alpha \min\{0, z\}$, donde α es un valor pequeño y positivo. La pendiente en la región negativa permite que la unidad reciba gradiente aun cuando su activación sea negativa.
- **Tanh:** $g(z) = \tanh(z)$, con valores de salida en $[-1, 1]$. Es simétrica respecto del origen, pero puede saturarse para valores extremos y dificultar el aprendizaje basado en gradientes.

Las redes *feedforward* organizan sus unidades en capas acíclicas. Cada capa recibe la salida de la anterior, aplica una transformación afín y, salvo cuando la tarea requiere una salida lineal, una función de activación. Para una red con L capas parametrizadas, esta operación puede escribirse como:

$$\mathbf{x}_k = g_k(\mathbf{W}_k^\top \mathbf{x}_{k-1} + \mathbf{b}_k), \quad k = 1, \dots, L, \quad \mathbf{x}_0 = \mathbf{x} \quad (2.13)$$

Aquí, $\mathbf{W}_k$ y $\mathbf{b}_k$ son los parámetros de la capa k , $\mathbf{x}_{k-1}$ es la salida de la capa anterior y g_k se aplica componente a componente. La arquitectura suele describirse mediante una capa de entrada, una o más capas ocultas y una capa de salida, aunque el número de capas se cuenta de acuerdo con la convención adoptada para los parámetros entrenables [BB24, GBC16]. La ecuación 2.13 resume la composición de funciones que permite construir una red profunda.

La Figura 2.11 representa esta organización para una red con dos capas ocultas. Las flechas avanzan únicamente hacia la derecha: cada unidad de una capa recibe las salidas de la capa anterior y las transforma mediante los parámetros de esa capa, sin conexiones recurrentes.

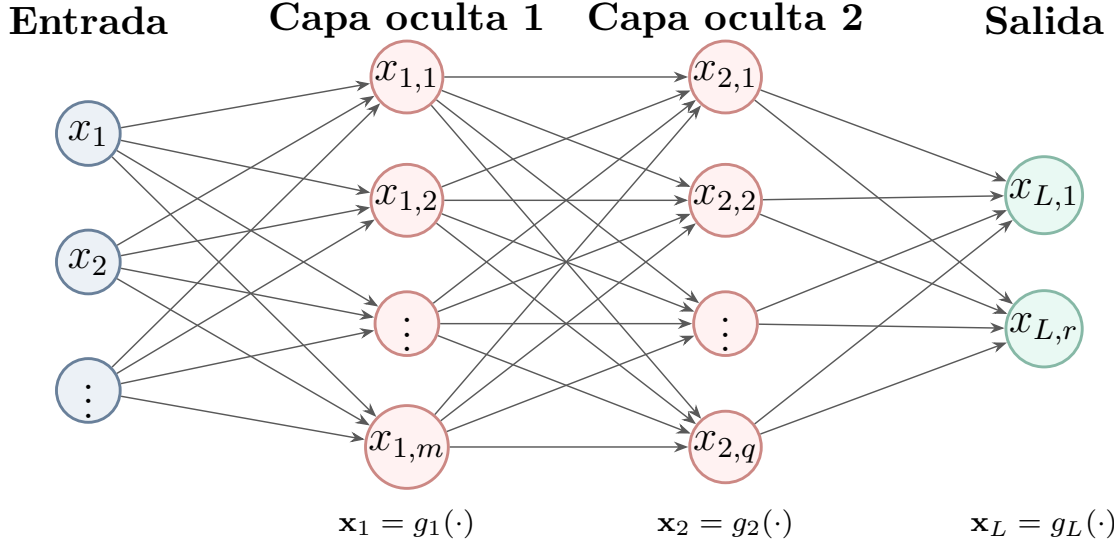


Figura 2.11: Arquitectura conceptual de una red neuronal *feedforward*. Cada columna representa una capa y las conexiones se dirigen desde la entrada hacia la salida; las expresiones inferiores indican la transformación afín seguida de la activación en cada capa.

El Teorema de Aproximación Universal es un resultado de existencia: bajo condiciones apropiadas, una red de dos capas parametrizadas, con una capa oculta suficientemente ancha, puede aproximar una función continua sobre un dominio compacto con una precisión arbitraria. El teorema no proporciona por sí mismo un procedimiento para encontrar los parámetros, no garantiza un entrenamiento eficiente y tampoco implica una buena generalización sobre datos no observados. La profundidad puede ofrecer representaciones jerárquicas y una mayor eficiencia representacional para ciertas familias de funciones, pero no garantiza por sí sola un menor error de prueba [BB24].

El Proceso de Optimización

Dado un conjunto de datos y una función de pérdida diferenciable $\mathcal{L}(\theta)$, los parámetros θ se ajustan mediante un procedimiento de optimización iterativa. En términos generales, el descenso de gradiente actualiza los parámetros según:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad (2.14)$$

donde η es la tasa de aprendizaje. En la práctica, el gradiente puede calcularse sobre todo el conjunto de entrenamiento, sobre una observación individual o sobre un mini-lote; el tamaño del mini-lote modifica el costo computacional y la variabilidad de la estimación,

pero no constituye un equilibrio óptimo garantizado [BB24].

- **Función de pérdida (*loss function*):** cuantifica la discrepancia entre las predicciones de la red y los valores objetivo. Puede adoptar formas distintas según la tarea; por ejemplo, el error cuadrático medio en regresión o la entropía cruzada en clasificación. En DQN, la pérdida se formula a partir del error de diferencia temporal presentado más adelante.
- **Descenso por mini-lotes:** utiliza un subconjunto de datos para estimar el gradiente en cada actualización. El barajado de las observaciones puede reducir correlaciones debidas al orden de los datos, pero el procedimiento no garantiza por sí mismo estabilidad ni convergencia.
- **Adam:** adapta la tasa de aprendizaje de cada parámetro utilizando promedios móviles del gradiente y de su magnitud al cuadrado. Su eficacia es principalmente empírica y depende de la función de pérdida, los datos y la configuración del entrenamiento; no constituye una garantía general de convergencia [BB24].
- **Retropropagación (*backpropagation*):** calcula eficientemente las derivadas parciales de la pérdida respecto de los pesos y sesgos mediante la regla de la cadena. No es el optimizador: proporciona los gradientes que luego utilizan el descenso de gradiente, Adam u otros métodos de actualización [BB24].

2.2.4 Aprendizaje por Refuerzo Profundo (DRL)

El Aprendizaje por Refuerzo Profundo (DRL) combina los algoritmos de RL con aproximadores neuronales. En el control semafórico, las observaciones del tránsito, como colas, ocupación o velocidades, pueden alimentar la red; sus salidas pueden representar valores de acción o parámetros de una política; y las recompensas asociadas con demoras, detenciones y colas orientan la función de pérdida del algoritmo de aprendizaje. Esta integración resulta útil cuando el espacio de estados es continuo o de alta dimensionalidad, aunque exige mecanismos de estabilización y una evaluación cuidadosa de la eficiencia de las muestras [ACS24].

El Problema de la Tríada Mortal (*Deadly Triad*)

La aplicación de aproximadores de funciones al aprendizaje por refuerzo introduce dificultades de estabilidad. Sutton y Barto [SB18] denominan *Tríada Mortal* a la combinación de tres elementos que puede producir inestabilidad o divergencia en las estimaciones de valor, especialmente cuando no se cumplen condiciones suficientes de representación y actualización:

- **Aproximación de Funciones:** Uso de aproximadores potentes y escalables, como las redes neuronales, que extienden el aprendizaje generalizando a través del espacio de estados.

- **Bootstrapping:** La práctica de actualizar estimaciones de valor apoyándose en otras estimaciones aprendidas previamente (como en los métodos TD), en lugar de esperar la consecución de retornos reales completos.
- **Entrenamiento *Off-Policy*:** Optimizar la función de valor utilizando una distribución de datos o transiciones generadas por una política de comportamiento distinta a la política objetivo que se desea evaluar o mejorar.

La tríada no implica divergencia inevitable ni obliga a conservar sus tres componentes. La aproximación de funciones suele ser necesaria para escalar a espacios de estados grandes o continuos, mientras que el *bootstrapping* y el aprendizaje *off-policy* pueden evitarse mediante métodos Monte Carlo y métodos *on-policy*, respectivamente. Estos intercambios modifican la varianza, el sesgo y la eficiencia de muestras del aprendizaje; por ello, los algoritmos profundos incorporan mecanismos de estabilización en lugar de asumir que la divergencia es inevitable [SB18].

Deep Q-Networks (DQN)

En los métodos basados en valor, DRL sustituye la tabla clásica por una red neuronal $Q(s, a; \theta)$. En espacios de acción discretos, el algoritmo *Deep Q-Networks* (DQN) procesa el estado s como vector de entrada y computa simultáneamente las estimaciones de valor para todas las acciones en su capa de salida [MKS⁺15].

La inestabilidad descrita por la Tríada Mortal se manifiesta en DQN como el “problema del blanco móvil”: al generalizar, una actualización que incrementa el valor de un estado puede modificar también los valores estimados para otros estados. Además, las transiciones consecutivas presentan correlación temporal y pueden inducir sobreajuste a la experiencia reciente. Para mitigar estos efectos y estabilizar empíricamente la optimización, DQN emplea dos componentes estructurales [MKS⁺15]:

- **Redes Objetivo (*Target Networks*):** Se emplea una red neuronal secundaria separada cuyos parámetros θ^- se congelan temporalmente. Esta red se utiliza exclusivamente para calcular los valores objetivo en el error TD, amortiguando las oscilaciones y estabilizando el blanco de aprendizaje.
- **Búferes de Repetición (*Replay Buffers*):** Se almacenan las transiciones históricas del agente (s, a, r, s') . El entrenamiento se realiza extrayendo mini-lotes aleatorios de esta memoria, lo cual reduce la correlación temporal intrínseca de las trayectorias y aproxima el entrenamiento a la hipótesis de muestras independientes. Este mecanismo no constituye, por sí mismo, una garantía de convergencia con aproximadores no lineales.

La Figura 2.12 distingue el circuito de interacción del circuito de entrenamiento. Durante la interacción, la red en línea transforma el estado del tránsito en un valor para

cada fase disponible y una regla ϵ -greedy selecciona la acción. La transición resultante se almacena en el búfer. Durante el entrenamiento, un mini-lote alimenta tanto la predicción de la red en línea como el cálculo del objetivo mediante la red objetivo; solo los parámetros θ se modifican en cada paso, mientras que θ^- se reemplaza periódicamente por una copia de θ .

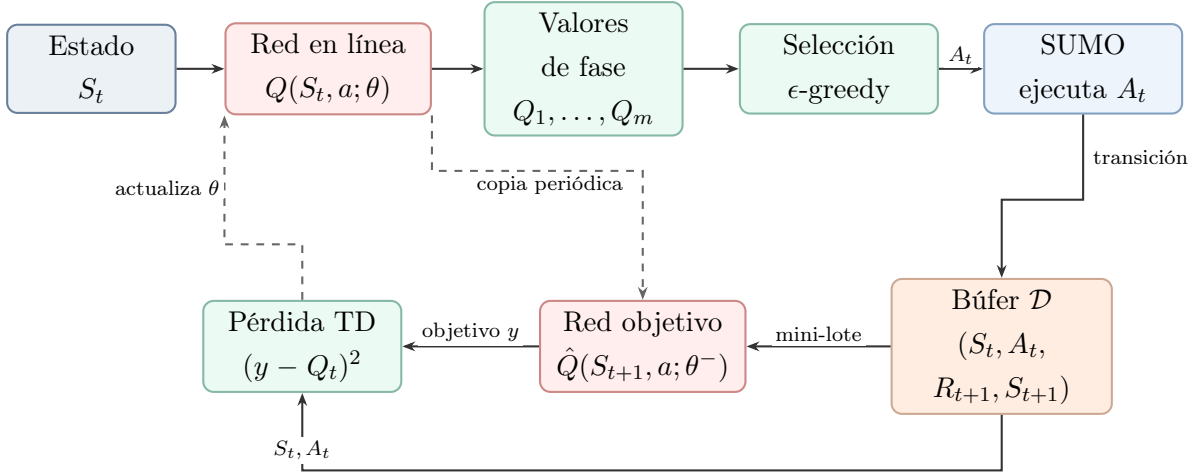


Figura 2.12: Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del blanco TD. Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS⁺15].

La función de pérdida minimizada por DQN, representada en el bloque inferior de la Figura 2.12, es:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1}) \sim \mathcal{D}} \left[\left(\underbrace{r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)}_{\text{Valor objetivo de TD}} - \underbrace{Q(s_j, a_j; \theta)}_{\text{Predicción actual}} \right)^2 \right] \quad (2.15)$$

donde $\mathcal{D}$ es el búfer de repetición de experiencias, $(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1})$ representa una transición muestreada, d_{j+1} vale uno cuando la transición es terminal y cero en caso contrario, θ indica los parámetros de la red Q actual y θ^- representa los parámetros congelados de la red objetivo.

El procedimiento conceptual de DQN combina la recolección de transiciones, la selección de acciones mediante una política ϵ -greedy y la actualización de la red a partir de mini-lotes extraídos del búfer de experiencias. El Algoritmo 1 resume este ciclo sin imponer decisiones particulares de implementación:

Algorithm 1 Deep Q-Network con repetición de experiencias

```
1: Inicializar el búfer de experiencias  $\mathcal{D}$  con capacidad  $N$ 
2: Inicializar la red de acción-valor  $Q(s, a; \theta)$  con parámetros aleatorios
3: Inicializar la red objetivo  $\hat{Q}(s, a; \theta^-)$  con  $\theta^- \leftarrow \theta$ 
4: for episodio = 1, 2, ... do
5:   Inicializar el estado  $s_t$ 
6:   for  $t = 1, 2, \dots, T$  do
7:     Seleccionar  $a_t$  mediante una política  $\epsilon$ -greedy basada en  $Q(s_t, a; \theta)$ 
8:     Ejecutar  $a_t$  y observar la recompensa  $r_{t+1}$ , el estado  $s_{t+1}$  y si el episodio terminó
9:     Almacenar  $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$  en  $\mathcal{D}$ 
10:    Muestrear un mini-lote de transiciones desde  $\mathcal{D}$ 
11:    Calcular  $y_j = r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$ 
12:    Actualizar  $\theta$  minimizando  $(y_j - Q(s_j, a_j; \theta))^2$ 
13:    Actualizar periódicamente la red objetivo:  $\theta^- \leftarrow \theta$ 
14:     $s_t \leftarrow s_{t+1}$ 
15:   end for
16: end for
```

En esta expresión, d_{j+1} es un indicador que toma el valor uno cuando la transición conduce a un estado terminal y cero en caso contrario.

Aproximación de Políticas, Función de Ventaja y *Policy Gradient*

Como alternativa a derivar la política de forma indirecta a partir de una función de valor (como en DQN), DRL permite parametrizar directamente la política del agente mediante una red neuronal $\pi(a|s; \theta)$ [ACS24]. En un problema de acciones discretas, la red recibe el estado u observación, calcula un puntaje para cada acción y transforma esos puntajes en una distribución de probabilidad, habitualmente mediante una función *softmax*. La acción puede muestrearse de esa distribución durante el entrenamiento, preservando la exploración estocástica.

La Figura 2.13 muestra este recorrido para un controlador semafórico. A diferencia de DQN, las salidas no son estimaciones de valor: son probabilidades que suman uno. La señal de ventaja no entra a la red para seleccionar la fase; se utiliza durante el entrenamiento para modificar los parámetros en la dirección indicada por el gradiente de la log-probabilidad de la acción observada.

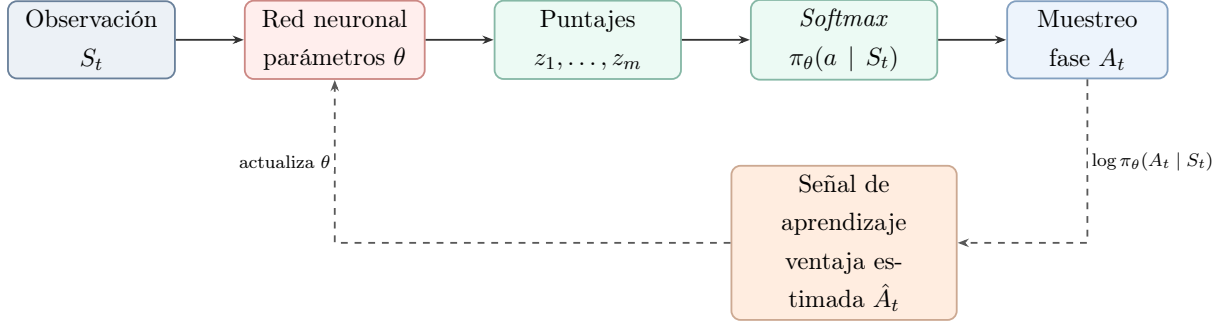


Figura 2.13: Red de política para un espacio discreto de fases. La observación determina una distribución de acciones; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03].

Para cuantificar el desempeño marginal de una acción tomada en un estado específico respecto a la expectativa promedio de la política, se define la **Función de Ventaja** (*Advantage Function*) $A^\pi(s, a)$ [SB18]:

$$A^\pi(s, a) \doteq Q^\pi(s, a) - V^\pi(s) \quad (2.16)$$

donde $Q^\pi(s, a)$ representa el retorno esperado al tomar la acción a en el estado s bajo la política π , y $V^\pi(s)$ es el valor promedio de estado. Conceptualmente, la ventaja mide cuánto mejor ($A^\pi(s, a) > 0$) o peor ($A^\pi(s, a) < 0$) resulta la acción a en comparación con el desempeño medio del agente en el estado s .

En arquitecturas actor-crítico, la ventaja se estima habitualmente mediante la Estimación Generalizada de la Ventaja (*Generalized Advantage Estimation*, GAE) desarrollada por Schulman et al. [SML⁺16]:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} \doteq \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \quad (2.17)$$

donde $\delta_t^V = R_{t+1} + \gamma(1 - d_{t+1})V(S_{t+1}) - V(S_t)$ representa el error de diferencia temporal (TD) estimado por la red del crítico V , d_{t+1} indica si la transición es terminal, $\gamma \in [0, 1]$ es el factor de descuento y $\lambda \in [0, 1]$ es el hiperparámetro de suavizado GAE que equilibra el sesgo y la varianza del estimador.

En lugar de minimizar un error cuadrático de predicción, la optimización paramétrica del actor se rige por el Teorema del Gradiente de Política (*Policy Gradient Theorem*), que establece la actualización de los parámetros θ ascendiendo por el gradiente del rendimiento esperado [SB18]. El operador $\hat{\mathbb{E}}_t$ denota el promedio empírico sobre las transiciones del lote recolectado en los instantes t :

$$\nabla_\theta J(\theta) = \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(A_t | S_t) \hat{A}_t \right] \quad (2.18)$$

donde $\nabla_\theta J(\theta)$ representa el gradiente del desempeño esperado, $\pi_\theta(A_t | S_t)$ es la probabilidad

idad otorgada a la acción ejecutada A_t en el estado S_t , y $\hat{A}_t$ es la ventaja estimada. Como ilustra la Figura 2.13, el término $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ indica cómo modificar los parámetros para cambiar la probabilidad de la acción observada, mientras que $\hat{A}_t$ determina el signo y la magnitud de esa modificación. Por lo tanto, la Ecuación 2.18 incrementa la probabilidad de elegir acciones con ventaja positiva y reduce la de aquellas con ventaja negativa.

Proximal Policy Optimization (PPO) y control de las actualizaciones

Una dificultad conocida de los métodos de gradiente de política tradicionales (como REINFORCE o A2C) es su sensibilidad a la tasa de aprendizaje: actualizaciones de parámetros con pasos demasiado grandes pueden modificar la distribución de la política de forma abrupta, provocando un colapso del desempeño del cual el agente raramente se recupera.

Para limitar el riesgo de actualizaciones demasiado grandes, Kakade y Langford [KL02] y Schulman et al. (TRPO [SLA⁺15]) estudiaron métodos de región de confianza. Bajo supuestos específicos y una optimización que respeta la restricción de divergencia entre políticas, el procedimiento teórico de TRPO puede establecer una mejora monótona del retorno:

$$J(\pi_{k+1}) \geq J(\pi_k) \quad (2.19)$$

donde $J(\pi_k)$ representa el desempeño esperado de la política en la iteración k . TRPO implementa esta idea mediante una restricción explícita sobre la divergencia Kullback–Leibler (KL). Por su parte, *Proximal Policy Optimization* (PPO) [SWD⁺17] aproxima el comportamiento de una región de confianza mediante un objetivo sustituto recortado (*clipped surrogate objective*) computable con métodos de primer orden. El recorte limita cambios excesivos en la política, pero no constituye una garantía estricta de mejora monótona en cada actualización.

Dada la razón de probabilidad entre la política actual y la anterior:

$$r_t(\theta) \doteq \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (2.20)$$

donde $\pi_{\theta}(a_t | s_t)$ es la probabilidad de la acción a_t bajo los nuevos parámetros θ , y $\pi_{\theta_{\text{old}}}(a_t | s_t)$ es la probabilidad registrada durante la recolección del lote, PPO maximiza el siguiente objetivo recortado:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.21)$$

donde $\epsilon > 0$ es un hiperparámetro de recorte (típicamente $\epsilon \approx 0.1\text{--}0.2$), y $\text{clip}(r, 1 - \epsilon, 1 + \epsilon)$ acota la razón r al intervalo $[1 - \epsilon, 1 + \epsilon]$. El operador $\min$ toma el valor más pesimista entre la ventaja ponderada simple y la ventaja recortada, eliminando el incentivo a realizar actualizaciones que modifiquen excesivamente la política si la ventaja es positiva,

y acotando la penalización si la ventaja es negativa.

En la práctica, el objetivo del actor de PPO se combina con una pérdida de valor y, opcionalmente, con un término de entropía para favorecer la exploración. Si se expresa todo como un objetivo a maximizar, una forma compacta es:

$$J^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 \hat{\mathbb{E}}_t \left[(V_\phi(s_t) - V_t^{\text{target}})^2 \right] + c_2 \hat{\mathbb{E}}_t [S[\pi_\theta](s_t)] \quad (2.22)$$

donde ϕ representa los parámetros de la red del crítico V_ϕ , V_t^{target} es el retorno objetivo estimado para ajustar el crítico, $S[\pi_\theta]$ denota la entropía de la distribución de la política, c_1 pondera el error de valor y c_2 pondera el incentivo a conservar exploración, con $c_1, c_2 > 0$.

PPO es un algoritmo predominantemente *on-policy* que reutiliza de forma limitada el lote recolectado con la política vigente. En esta investigación constituye la base conceptual de IPPO (*Independent Proximal Policy Optimization*), MAPPO (*Multi-Agent Proximal Policy Optimization*) y HAPPO (*Heterogeneous-Agent Proximal Policy Optimization*); las diferencias entre estas variantes se explican en la sección multiagente [YVV⁺22, KCW⁺22].

El Algoritmo 2 resume el ciclo conceptual de PPO. La recolección de trayectorias se realiza con la política vigente y el lote se reutiliza durante un número limitado de épocas; posteriormente se descarta para obtener nuevas muestras con la política actualizada:

Algorithm 2 Proximal Policy Optimization (PPO)

- 1: Inicializar los parámetros del actor θ y del crítico ϕ
 - 2: **for** iteración = 1, 2, ... **do**
 - 3: Recolectar trayectorias con la política $\pi_{\theta_{\text{old}}}$
 - 4: Calcular los retornos objetivo y las ventajas estimadas $\hat{A}_t$
 - 5: **for** época = 1 hasta K **do**
 - 6: Muestrear mini-lotes del conjunto de trayectorias recolectado
 - 7: Calcular $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
 - 8: Actualizar θ maximizando $L^{\text{CLIP}}(\theta)$
 - 9: Actualizar ϕ minimizando el error cuadrático del valor
 - 10: Incorporar, si corresponde, un término de entropía para favorecer la exploración
 - 11: **end for**
 - 12: $\theta_{\text{old}} \leftarrow \theta$
 - 13: **end for**
-

2.3 Aprendizaje por Refuerzo Multi-Agente (MARL)

El aprendizaje por refuerzo multi-agente (*Multi-Agent Reinforcement Learning*, MARL) generaliza los principios del RL hacia sistemas donde múltiples entidades aprenden e interactúan simultáneamente en un entorno compartido [ACS24]. A diferencia del caso

individual, las acciones de cada agente modifican el estado global del sistema, lo que induce una dinámica no estacionaria desde la perspectiva local de cualquier participante. En el contexto de sistemas dinámicos cooperativos, múltiples controladores coordinan sus decisiones para optimizar un objetivo común.

En una intersección aislada, el control puede analizarse mediante un único agente. Sin embargo, en una red vial la fase seleccionada en una intersección modifica las corrientes que llegan a otras, por lo que los controladores quedan acoplados por la dinámica del tránsito. Esta dependencia espacial motiva la extensión del aprendizaje por refuerzo hacia un escenario multi-agente.

2.3.1 Fundamentos Teóricos: Juegos Estocásticos, Equilibrio de Nash y Dec-POMDP

La formalización matemática de los entornos multi-agente requiere extender el MDP individual incorporando nociones de la Teoría de Juegos. El modelo matemático fundamental para representar decisiones secuenciales con múltiples participantes es el Juego Estocástico o Juego Markoviano (*Stochastic Game* o *Markov Game*), introducido por Shapley [Sha53] y adaptado al aprendizaje por refuerzo por Albrecht et al. [ACS24].

Un Juego Estocástico para n agentes se define mediante la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, \{R_i\}_{i=1}^n, \gamma \rangle \quad (2.23)$$

donde $\mathcal{N} = \{1, \dots, n\}$ representa el conjunto de agentes, $\mathcal{S}$ es el espacio de estados globales, $\mathcal{A}_i$ es el espacio de acciones del agente i , $P(s' \mid s, \mathbf{a})$ representa la función de transición de estados condicionada a la acción conjunta $\mathbf{a} = (a_1, \dots, a_n) \in \mathcal{A} \equiv \mathcal{A}_1 \times \dots \times \mathcal{A}_n$, y $R_i(s, \mathbf{a})$ representa la función de recompensa asignada al agente i . En un entorno estrictamente cooperativo, todos los agentes comparten exactamente la misma señal de recompensa ($R_1 = R_2 = \dots = R_n = R$), de modo que el objetivo de cada agente es maximizar el retorno global esperado del sistema.

En el control semafórico de una red, el conjunto de agentes puede corresponder a las intersecciones controladas, mientras que cada espacio de acciones contiene las fases válidas de una intersección. El estado global y la transición representan la evolución conjunta de la red, que puede ser simulada en SUMO, y R_i resume la señal de aprendizaje asociada con el desempeño del controlador i .

En la teoría de juegos, el **Equilibrio de Nash** es un concepto de solución que caracteriza la estabilidad frente a desviaciones unilaterales [ACS24]. Una política conjunta $\boldsymbol{\pi}^* = (\pi_1^*, \dots, \pi_n^*)$ constituye un Equilibrio de Nash si ningún agente $i \in \mathcal{N}$ puede incrementar unilateralmente su retorno esperado desviándose de su política π_i^* , asumiendo que

los demás agentes mantienen sus políticas fijas en $\boldsymbol{\pi}_{-i}^*$:

$$J(\pi_i, \boldsymbol{\pi}_{-i}^*) \leq J(\pi_i^*, \boldsymbol{\pi}_{-i}^*), \quad \forall \pi_i \in \Pi_i, \quad \forall i \in \mathcal{N} \quad (2.24)$$

donde $\boldsymbol{\pi}^* = (\pi_1^*, \dots, \pi_n^*)$ es la combinación de políticas individuales en equilibrio, π_i representa cualquier política alternativa ejecutable por el agente i , Π_i denota el espacio de políticas válidas del agente i , y $\boldsymbol{\pi}_{-i}^* = (\pi_1^*, \dots, \pi_{i-1}^*, \pi_{i+1}^*, \dots, \pi_n^*)$ es el vector de políticas fijas de los demás agentes. En un problema cooperativo de recompensa común, este concepto describe estabilidad, pero no garantiza por sí mismo que la política conjunta maximice el retorno global. Por esa razón, el objetivo operativo de este trabajo se formula como la maximización del retorno conjunto, mientras que el equilibrio de Nash se utiliza como concepto de análisis de convergencia.

En escenarios reales de control distribuido, ningún agente dispone de visibilidad completa del estado global del sistema. Cada participante recopila información únicamente de sus sensores locales. Por consiguiente, los entornos cooperativos multi-agente con observabilidad parcial se formalizan matemáticamente mediante el marco de los Procesos de Decisión de Markov Parcialmente Observables Descentralizados (Dec-POMDP) [AOS20].

Un Dec-POMDP para n agentes se define como una tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, R, \{\Omega_i\}_{i=1}^n, O, \gamma \rangle \quad (2.25)$$

donde Ω_i denota el conjunto de observaciones individuales del agente i , regidas por la función de observación probabilística $O(o_1, \dots, o_n \mid s', \mathbf{a}) = \Pr\{O_1 = o_1, \dots, O_n = o_n \mid S' = s', \mathbf{A} = \mathbf{a}\}$. En este esquema, cada agente i basa sus decisiones en su observación local $o_i \in \Omega_i$ o en una representación de su historial de observaciones, con el propósito de aprender una política descentralizada $\pi_i(a_i \mid o_i)$ que, al combinarse con las políticas de los demás agentes en la política conjunta $\boldsymbol{\pi} = (\pi_1, \dots, \pi_n)$, maximice el retorno acumulado conjunto del sistema [AOS20].

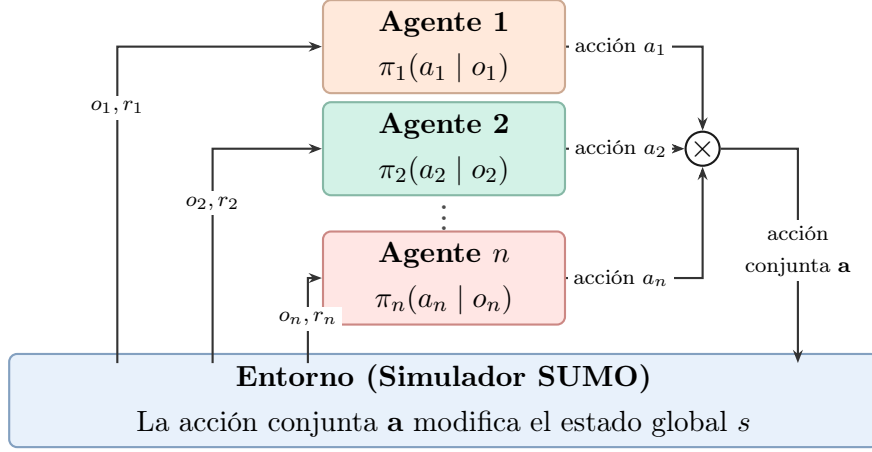


Figura 2.14: Interacción Agente-Entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.

La Figura 2.14 muestra cómo las acciones locales de los controladores se combinan antes de modificar el estado global de la red.

2.3.2 Desafíos Fundamentales en MARL

La aplicación práctica de métodos de aprendizaje por refuerzo en escenarios multi-agente enfrenta obstáculos analíticos y computacionales significativos que no existen en el caso individual [ACS24]. A medida que aumenta el número de agentes, el espacio de acciones conjunto $\mathcal{A}$ crece de forma exponencial con el número de agentes ($|\mathcal{A}| = \prod_{i=1}^n |\mathcal{A}_i|$), lo que introduce un crecimiento combinatorio del espacio de decisiones. Sin embargo, los desafíos teóricos más severos radican en la dinámica propia de la interacción:

- **No Estacionariedad:** Desde la perspectiva de un agente individual i , las políticas de los demás agentes actúan como parte de la dinámica de transición del entorno. Como todos los agentes aprenden y adaptan sus pesos neuronales de manera concurrente, la función de transición efectiva percibida localmente por el agente i :

$$\mathcal{P}_i(s' | s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' | s, a_i, \mathbf{a}_{-i}) \boldsymbol{\pi}_{-i}(\mathbf{a}_{-i} | s) \quad (2.26)$$

cambia a lo largo del entrenamiento. Aquí $\mathbf{a}_{-i}$ representa las acciones conjuntas del resto de los agentes y $\boldsymbol{\pi}_{-i}$ sus respectivas políticas. Aunque el sistema global puede continuar describiéndose mediante un juego markoviano, la dinámica percibida por un agente local se vuelve no estacionaria cuando las políticas de los demás agentes cambian y no forman parte de su observación. En consecuencia, las condiciones de convergencia de los algoritmos de agente único dejan de ser suficientes [FNF⁺17].

- **Observabilidad Parcial:** Dado el modelo Dec-POMDP inherente, la falta de telemetría global restringe la visión de cada agente a variables locales. Esta in-

formación incompleta aumenta la incertidumbre de las estimaciones y dificulta el aprendizaje preciso de las funciones de valor.

- **Asignación de Crédito Multi-Agente (*Credit Assignment*):** En juegos de recompensa común, resulta complejo discernir analíticamente la contribución o el demérito marginal que la acción de un agente aislado produjo sobre el desempeño sistémico global. Una decisión local subóptima podría recibir un refuerzo positivo si el resto del sistema operó de manera sumamente eficiente, sesgando las actualizaciones de la política [ACS24].

2.3.3 Paradigmas de Entrenamiento y Ejecución

Para abordar estas patologías en escenarios Dec-POMDP, la literatura clasifica los algoritmos en función de cómo fluye la información durante el proceso de optimización. Se distinguen fundamentalmente dos enfoques: el Aprendizaje Independiente (*Independent Learning*, IL) y el Entrenamiento Centralizado con Ejecución Descentralizada (*Centralized Training with Decentralized Execution*, CTDE) [ZLYZ23].

En el primero, los agentes se entrenan localmente utilizando métodos de agente único, tratando empíricamente al resto de los agentes como parte de la dinámica del entorno. En el segundo, durante el entrenamiento, los algoritmos pueden explotar información global privilegiada (historiales conjuntos o concatenación de observaciones) para guiar y estabilizar la optimización de las funciones de valor. Durante la ejecución, el componente centralizado no participa en la selección de acciones y los actores utilizan únicamente sus observaciones locales. Esta separación entre información disponible durante el entrenamiento y durante la ejecución constituye la característica central de CTDE.

En la implementación empleada, esta separación se concreta mediante un crítico compartido que procesa las observaciones de los agentes durante el entrenamiento, mientras que cada actor conserva su observación local para seleccionar acciones durante la ejecución.

2.3.4 MADRL y líneas de base

La integración de los formalismos Dec-POMDP con la capacidad de aproximación generalizada del aprendizaje profundo ha dado origen al campo del *Multi-Agent Deep Reinforcement Learning* (MADRL) [LWT⁺17]. Esta área agrupa los enfoques prevalentes para el control adaptativo de sistemas complejos a gran escala.

En esta investigación, para evaluar rigurosamente el desempeño de las arquitecturas de entrenamiento centralizado (MAPPO y HAPPO), resulta metodológicamente conveniente establecer **líneas de base de aprendizaje independiente (IL)**. En los algoritmos independientes, cada controlador actúa como un agente aislado que no modela la presencia explícita de los demás participantes.

Aprendizaje Independiente (IL): IDQN e IPPO como Líneas de Base

- **Independent DQN (IDQN) como línea de base basada en valor:** Representa una extensión directa de DQN en la que cada agente mantiene su propia función de acción-valor [TMK⁺17]. Como método *off-policy*, puede reutilizar transiciones históricas mediante un *replay buffer*; en MARL, esas transiciones fueron generadas mientras los demás agentes seguían políticas posiblemente distintas. Este desfase puede agravar la no estacionariedad y las dificultades de estabilidad del aprendizaje basado en valor [FNF⁺17]. En la evaluación experimental de este trabajo, IDQN sirve como línea de base independiente para los métodos basados en valor.
- **Independent PPO (IPPO) como línea de base basada en política:** Asigna una instancia de PPO a cada agente. Al utilizar lotes recolectados con las políticas vigentes y descartar la memoria histórica acumulativa, reduce la exposición a transiciones generadas bajo políticas antiguas, aunque no elimina la no estacionariedad causada por el aprendizaje simultáneo [dWGM⁺20]. El recorte de PPO limita cambios excesivos en cada actualización, pero no ofrece una garantía general de convergencia. IPPO constituye la línea de base independiente principal frente a los algoritmos coordinados.

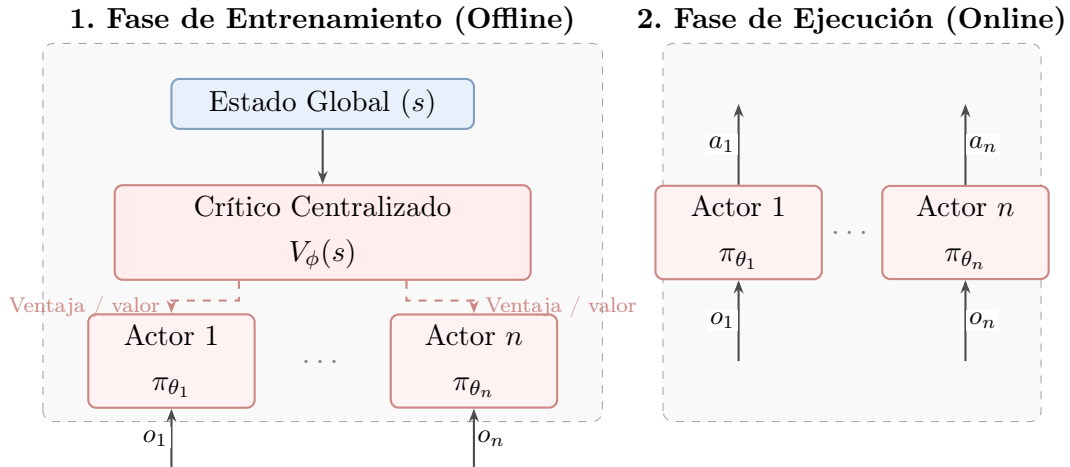


Figura 2.15: Paradigma de Entrenamiento Centralizado y Ejecución Descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.

2.3.5 Multi-Agent Proximal Policy Optimization (MAPPO)

Para estudiar cómo una configuración de PPO puede funcionar en tareas cooperativas, Yu et al. [YVV⁺22] evaluaron empíricamente la variante conocida como *Multi-Agent Proximal Policy Optimization* (MAPPO), adaptando PPO al entorno multiagente bajo el paradigma CTDE (véase la Figura 2.15). Su contribución principal fue sistematizar prácticas de

implementación y evaluar su efecto, no proponer desde cero un algoritmo independiente de PPO.

En MAPPO, la estructura distingue entre actores descentralizados y un crítico centralizado:

- **Actores descentralizados** ($\pi_{\theta_i}(a_i | o_i)$): Cada agente i mantiene una política que procesa su observación local o_i y produce una distribución sobre las acciones locales a_i . En entornos homogéneos puede utilizarse compartición de parámetros para mejorar la eficiencia de aprendizaje, pero esta práctica no constituye un requisito conceptual de MAPPO.
- **Crítico centralizado** ($V_\phi(s)$): Durante el entrenamiento, una red de función de valor puede recibir el estado global s del entorno o una representación construida a partir de observaciones conjuntas. El crítico estima el retorno y proporciona valores para calcular ventajas mediante GAE; durante la ejecución no es necesario disponer de este componente [YVV⁺22].

En la formulación práctica de MAPPO, el objetivo del actor se expresa como un promedio de términos PPO individuales. La política conjunta se factoriza en políticas locales, pero la pérdida no debe interpretarse como una única razón de probabilidad conjunta:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t(s_t, \mathbf{a}_t), \text{clip}(r_t^i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t(s_t, \mathbf{a}_t) \right) \right] \quad (2.27)$$

donde $r_t^i(\theta) = \frac{\pi_{\theta_i}(a_t^i | o_t^i)}{\pi_{\theta_{i,\text{old}}}(a_t^i | o_t^i)}$ es la razón de probabilidad individual del agente i , n es el número total de agentes, $\hat{A}_t$ es la ventaja utilizada por la implementación y ϵ es el hiperparámetro de recorte. En entornos con recompensa común, esta ventaja puede calcularse utilizando un crítico centralizado; la forma exacta depende de la definición del estado y de la implementación.

Por su parte, el crítico centralizado se actualiza minimizando el error cuadrático medio respecto al retorno objetivo descontado normalizado:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right] \quad (2.28)$$

donde ϕ son los parámetros del crítico centralizado y R_t^{target} representa el retorno empírico objetivo.

Factores Críticos de Diseño y Optimización en MAPPO

A pesar de su simplicidad conceptual, Yu et al. [YVV⁺22] demostraron empíricamente que la efectividad de MAPPO depende fuertemente de varias decisiones de diseño e hiperparámetros:

1. **Normalización del valor:** La normalización de los retornos puede mejorar el ajuste del crítico cuando la escala de las recompensas cambia durante el aprendizaje. Yu et al. analizan esta decisión junto con otras prácticas de implementación; PopArt es una alternativa posible, no una condición necesaria de MAPPO [YVV⁺22].
2. **Representación del Estado Global en el Crítico:** Yu et al. [YVV⁺22] evaluaron distintas opciones para estructurar la entrada s del crítico centralizado:
 - Concatenación de Observaciones Locales (*Concatenated Local, CL*): forma el estado uniendo las observaciones locales de todos los agentes $s = (o_1, \dots, o_n)$.
 - Estado Global del Entorno (*Environment Provided, EP*): utiliza un vector de estado global provisto directamente por el simulador.
 - Estado Específico por Agente (*Agent-Specific State, AS*): combina el estado del entorno con la información local del agente i ($AS_i = EP \cup o_i$).
 - Filtrado de Características Redundantes (*Feature Pruning, FP*): remueve variables locales duplicadas o de baja varianza en el vector AS_i .

El estudio encuentra que las representaciones específicas por agente y el filtrado de redundancias pueden ser útiles en determinados bancos de prueba. Esta evidencia debe trasladarse al escenario de tránsito mediante evaluación, no como una regla universal.

3. **Reutilización restringida de datos:** En los experimentos de Yu et al., pocas épocas y una o dos particiones del lote resultaron favorables en varios entornos cooperativos. Son recomendaciones empíricas sensibles a la dificultad del problema, no garantías de estabilidad para cualquier red vial.
4. **Rango de recorte y tamaño de lote:** Valores de ϵ inferiores a 0.2 y lotes suficientemente grandes tendieron a favorecer la estabilidad en sus experimentos. La selección final debe justificarse con la búsqueda hiperparamétrica de esta investigación.

El Algoritmo 3 resume la recolección de trayectorias, el cálculo centralizado de ventajas y la actualización de los actores locales y del crítico compartido.

Algorithm 3 Algoritmo *Multi-Agent Proximal Policy Optimization* (MAPPO)

- 1: Inicializar los parámetros de la política (actor) θ y del valor (crítico) ϕ
 - 2: Inicializar el búfer de datos $\mathcal{D} \leftarrow \emptyset$
 - 3: **for** iteración = 1, 2, ... **do**
 - 4: Recolectar trayectorias *on-policy* para todos los agentes utilizando la política conjunta π_θ
 - 5: Calcular las ventajas conjuntas $\hat{A}_t(s_t, \mathbf{a}_t)$ mediante GAE a partir del crítico centralizado $V_\phi(s_t)$
 - 6: Calcular los retornos objetivos R_t^{target} normalizados para el crítico
 - 7: Almacenar las transiciones $(s_t, o_t, \mathbf{a}_t, \hat{A}_t, R_t^{\text{target}})$ en $\mathcal{D}$
 - 8: **for** época = 1 hasta K **do**
 - 9: Muestrear minilotes de datos desde $\mathcal{D}$
 - 10: Actualizar la política maximizando el objetivo de recorte PPO por agente:
 $\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t, \text{clip}(r_t^i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$
 - 11: Actualizar el crítico ϕ minimizando el error cuadrático medio del valor:
 $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[(V_\phi(s_t) - R_t^{\text{target}})^2 \right]$
 - 12: **end for**
 - 13: Vaciar el búfer de datos $\mathcal{D} \leftarrow \emptyset$
 - 14: **end for**
-

2.3.6 HAPPO

En sistemas de control complejos, los agentes pueden presentar asimetrías físicas y operacionales: diferencias en la configuración de actuadores, distintos espacios de acciones o demandas regionales contrapuestas. En entornos homogéneos, compartir parámetros puede mejorar la eficiencia de aprendizaje; en entornos heterogéneos, esa práctica puede limitar la especialización. Además, las actualizaciones independientes no poseen una garantía general de convergencia debido al aprendizaje simultáneo no coordinado [KCW⁺22].

Kuba et al. [KCW⁺22] propusieron HATRPO y HAPPO como algoritmos que permiten políticas heterogéneas sin exigir compartición de parámetros ni una descomposición monótona de la función de valor conjunta. Cada agente i puede mantener su propia arquitectura y parámetros θ^i , adaptados a su espacio de acciones local $\mathcal{A}_i$. Los resultados teóricos de mejora monótona y convergencia se refieren al procedimiento de región de confianza bajo supuestos explícitos; la variante práctica HAPPO sustituye la restricción KL por un objetivo recortado y debe interpretarse como una aproximación [KCW⁺22].

Compartición de Parámetros y Políticas Heterogéneas

Una distinción central entre MAPPO y HAPPO radica en la homogeneidad de las políticas y en el orden de actualización:

- **Compartición de parámetros:** En MAPPO se utiliza habitualmente cuando los

agentes poseen espacios de observación y acción compatibles. Todos los agentes pueden compartir los mismos pesos $\theta_1 = \dots = \theta_n = \theta$, aunque la formulación de MAPPO también puede implementarse sin esta restricción.

- **Políticas heterogéneas y actualización secuencial:** HAPPO permite que cada agente mantenga una política independiente $\pi_{\theta^i}(a_i | o_i)$ y actualiza los actores siguiendo una permutación. El lote de trayectorias de la iteración actual se reutiliza de forma limitada; esto no equivale a un búfer histórico de *experience sharing* como el empleado por algoritmos específicos de compartición de experiencia.

Lema de Descomposición de la Ventaja Multi-Agente

El soporte matemático de HAPPO se fundamenta en el Lema de Descomposición de la Ventaja Multi-Agente propuesto por Kuba et al. [KCW⁺22].

En cualquier juego markoviano cooperativo, para una política conjunta π y una permutación ordenada arbitraria de los agentes $i_{1:n} = (i_1, i_2, \dots, i_n) \in \text{Sym}(n)$, la ventaja de una acción conjunta $\mathbf{a} = (a_{i_1}, \dots, a_{i_n})$ en el estado s se descompone exactamente como la suma de las ventajas marginales de cada agente tomadas secuencialmente, condicionadas por las acciones de los agentes que lo preceden en el ordenamiento:

$$A^\pi(s, \mathbf{a}) = \sum_{m=1}^n A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \quad (2.29)$$

donde $A^\pi(s, \mathbf{a})$ representa la ventaja conjunta global, y la ventaja marginal del agente i_m se define formalmente como:

$$A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \doteq Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) - Q_{i_{1:m-1}}^\pi(s, \mathbf{a}_{i_{1:m-1}}) \quad (2.30)$$

donde $Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m})$ representa el valor esperado de la acción conjunta considerando las acciones fijadas por los primeros m agentes en la secuencia, y $Q_{i_{1:m-1}}^\pi(s, \mathbf{a}_{i_{1:m-1}})$ es el valor marginalizado sobre los agentes restantes.

En esta formulación, las acciones de los agentes que aún no han actualizado su política en la secuencia $(\mathbf{a}_{i_{m+1:n}})$ se marginalizan por esperanza matemática bajo la política conjunta vigente. La gran relevancia teórica de este lema es que se cumple para cualquier juego cooperativo de recompensa común sin requerir ningún supuesto de aditividad o monotonía sobre la función de valor, a diferencia de los esquemas de descomposición de valor como VDN [SLG⁺18] o QMIX [RSDW⁺18].

Esquema de Actualización Secuencial y Factor de Razón Compuesto M

Para explotar la descomposición de la ventaja sin incurrir en la inestabilidad del entrenamiento simultáneo, HAPPO actualiza las políticas de los agentes de manera secuencial dentro de cada iteración de optimización k .

En cada iteración de entrenamiento k , se genera una permutación de agentes $i_{1:n} \in \text{Sym}(n)$. En el análisis teórico, asignar probabilidad no nula a los órdenes posibles es una condición que interviene en el resultado asintótico de convergencia; en una implementación finita, la aleatorización del orden reduce sesgos de prioridad, pero no constituye por sí sola una garantía [KCW⁺22].

Cuando le corresponde el turno de optimización al agente i_m , los agentes anteriores en la secuencia $(i_{1:m-1})$ ya han actualizado sus parámetros a $\theta_{k+1}^{i_j}$, mientras que los agentes posteriores $(i_{m+1:n})$ conservan sus parámetros originales $\theta_k^{i_j}$. Para guiar la actualización del agente i_m respetando la modificación sufrida por la distribución conjunta debido a los agentes precedentes, Kuba et al. [KCW⁺22] demostraron que no es necesario entrenar un crítico separado por cada subgrupo, sino que la ventaja marginal ajustada se puede calcular multiplicando la ventaja conjunta $A^{\pi_k}(s, \mathbf{a})$ por la productoria de los ratios de probabilidad de los agentes que ya se actualizaron.

Se define así la ventaja conjunta ponderada por los ratios de los agentes ya actualizados:

$$M_{i_{1:m}}(s, \mathbf{a}) \doteq \left(\prod_{j=1}^{m-1} \frac{\pi_{\theta_{k+1}^{i_j}}^{i_j}(a^{i_j} | o^{i_j})}{\pi_{\theta_k^{i_j}}^{i_j}(a^{i_j} | o^{i_j})} \right) A^{\pi_k}(s, \mathbf{a}) \quad (2.31)$$

Donde $\pi_{\theta_{k+1}^{i_j}}^{i_j}$ es la política recién optimizada del agente precedido i_j , $\pi_{\theta_k^{i_j}}^{i_j}$ es su política previa, y $A^{\pi_k}(s, \mathbf{a})$ es la ventaja conjunta. Para el primer agente de la secuencia ($m = 1$), la productoria es vacía y se define como $M_{i_1}(s, \mathbf{a}) \equiv A^{\pi_k}(s, \mathbf{a})$. Por tanto, M no es únicamente un cociente de probabilidades: es una señal de ventaja reponderada.

El factor $M_{i_{1:m}}(s, \mathbf{a})$ actúa como un modificador de importancia que ajusta la señal de ventaja para el agente i_m , absorbiendo analíticamente el desfase en la distribución de la acción conjunta provocado por las actualizaciones precedentes sin necesidad de realizar nuevas simulaciones en el entorno.

De este modo, para optimizar la política del agente i_m , HAPPO maximiza el objetivo sustituto recortado de PPO ponderado por la razón compuesta $M_{i_{1:m}}$:

$$L^{\text{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t [\min(r_t^{i_m}(\theta^{i_m}) M_{i_{1:m}}(s_t, \mathbf{a}_t), \text{clip}(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon, 1 + \epsilon) M_{i_{1:m}}(s_t, \mathbf{a}_t))] \quad (2.32)$$

donde $r_t^{i_m}(\theta^{i_m}) = \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ representa la razón de probabilidad individual del agente i_m .

Una vez realizada la actualización de descenso de gradiente sobre θ^{i_m} , es indispensable recalcular la razón de probabilidad post-optimización $\rho_{\text{post}}^{i_m} = \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ y actualizar recursivamente el factor M para los agentes restantes:

$$M_{i_{1:m+1}}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_{1:m}}(s_t, \mathbf{a}_t) \quad (2.33)$$

Garantía de Mejora Monótona

El trabajo de Kuba et al. demuestra una propiedad de mejora monótona para un procedimiento de iteración de políticas multiagente con restricciones de región de confianza y bajo supuestos explícitos [KCW⁺22]. La variante práctica HAPPO reemplaza la restricción KL por un objetivo recortado, por lo que esta garantía no se transfiere automáticamente a cada actualización de una implementación neuronal:

En el procedimiento teórico, la actualización secuencial satisface:

$$J(\boldsymbol{\pi}_{k+1}) \geq J(\boldsymbol{\pi}_k) \quad (2.34)$$

donde $J(\boldsymbol{\pi}_k)$ representa el rendimiento esperado de la política conjunta en el paso k . El resultado depende de las hipótesis del procedimiento analítico y no significa que HAPPO-clip elimine las oscilaciones ni que una ejecución finita con redes neuronales converja necesariamente a un equilibrio de Nash. En esta investigación, HAPPO se evalúa como una estrategia que busca mejorar la coordinación y la estabilidad frente a actualizaciones independientes.

La Figura 2.16 ilustra el orden secuencial de actualización que distingue a HAPPO de una actualización independiente simultánea. En ella, M representa una señal de ventaja reponderada por los cambios de los agentes precedentes, no un cociente de probabilidad aislado.

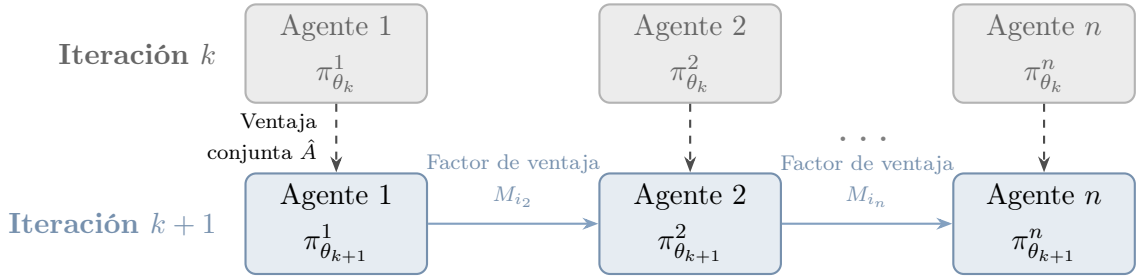


Figura 2.16: Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m} , que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.

El Algoritmo 4 resume la actualización secuencial de las políticas heterogéneas y la propagación recursiva del factor M .

Algorithm 4 Algoritmo *Heterogeneous-Agent Proximal Policy Optimization* (HAPPO)

- 1: Inicializar los parámetros de las políticas individuales $\{\theta^i, \forall i \in \mathcal{N}\}$ y del crítico centralizado $V_\phi(s)$
 - 2: **for** iteración = 1, 2, ... **do**
 - 3: Recolectar un conjunto de trayectorias $\mathcal{D}$ utilizando la política conjunta π_θ
 - 4: Calcular el estimador de ventaja conjunta $\hat{A}(s, \mathbf{a})$ con GAE a partir del crítico $V_\phi(s)$
 - 5: Sorteo aleatorio de una permutación de los agentes $i_{1:n} \in \text{Sym}(n)$
 - 6: Inicializar los factores $M_{i_m}(s, \mathbf{a}) \leftarrow \hat{A}(s, \mathbf{a})$ para todo $m \in \{1, \dots, n\}$
 - 7: **for** $m = 1 \dots n$ **do**
 - 8: Obtener el agente actual i_m y su factor modificador $M_{i_m}(s, \mathbf{a})$
 - 9: Actualizar θ^{i_m} a $\theta_{k+1}^{i_m}$ maximizando la pérdida recortada HAPPO:
$$L^{\text{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t \left[\min \left(r_t^{i_m}(\theta^{i_m}) M_{i_m}(s_t, \mathbf{a}_t), \text{clip} \left(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon, 1 + \epsilon \right) M_{i_m}(s_t, \mathbf{a}_t) \right) \right]$$
 - 10: Recalcular la razón post-optimización: $\rho_{\text{post}}^{i_m} \leftarrow \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$
 - 11: **for** $j = m + 1 \dots n$ **do**
 - 12: Actualizar el factor modificador para agentes subsecuentes:
$$M_{i_j}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_j}(s_t, \mathbf{a}_t)$$
 - 13: **end for**
 - 14: **end for**
 - 15: Actualizar el crítico centralizado ϕ mediante descenso de gradiente:
$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$$
 - 16: **end for**
-

Para finalizar el estudio comparativo de los algoritmos basados en Optimización de Política Próxima en escenarios MARL, la Tabla 2.3 sintetiza sus principales propiedades operativas, arquitectónicas y teóricas.

Tabla 2.3: Comparación sintética de familias algorítmicas PPO en entornos multi-agente cooperativos.

Eje de análisis	PPO de agente único	IPPO independiente	MAPPO / CTDE	HAPPO / HARL
Paradigma	Agente Único	IL (Descentralizado)	CTDE (Centralizado/Desc.)	CTDE / HARL
Actores	Único $\pi_\theta(a s)$	n políticas locales	Actores locales; puede compartir parámetros	n políticas heterogéneas
Arquitectura del Crítico	Único $V_\phi(s)$	n Críticos locales $V_{\phi_i}(o_i)$	Un Crítico Centralizado $V_\phi(s)$	Un Crítico Centralizado $V_\phi(s)$
Optimización	PPO estándar	PPO local simultáneo	PPO local con crítico global	Actualización secuencial $i_{1:n}$
Señal de Ventaja	Ventaja local $\hat{A}(s, a)$	Ventaja local $\hat{A}_i(o_i, a_i)$	Ventaja conjunta $\hat{A}(s, \mathbf{a})$	Factor compuesto $M_{i_{1:m}}(s, \mathbf{a})$
Mejora monótona	No estricta para PPO	Sin garantía general	Sin garantía estricta	Teórica bajo supuestos; HAPPO-clip es aproximado
Tipo de agente	No aplica	Depende de la implementación	Compatible; compartir parámetros es opcional	Diseñada para agentes heterogéneos

La correspondencia anterior permite interpretar IPPO como una referencia de aprendizaje independiente y MAPPO y HAPPO como arquitecturas que incorporan información conjunta durante el entrenamiento. El capítulo siguiente especifica cómo se instancian las observaciones, las acciones y las señales de recompensa en el entorno utilizado para la evaluación experimental.

Metodología y Diseño Experimental

En este capítulo se detalla el marco metodológico de la investigación, la operacionalización de las variables, el modelado físico y calibración de los escenarios de simulación, la arquitectura computacional del entorno distribuido y los protocolos experimentales adoptados para contrastar las hipótesis planteadas. El estudio se encuadra dentro de un enfoque cuantitativo, aplicado y experimental puro de medidas repetidas, donde la simulación microscópica de tránsito constituye el laboratorio controlado para evaluar el desempeño, la coordinación espacial y la robustez de las arquitecturas de Aprendizaje por Refuerzo Multi-Agente (MARL).

3.1 Enfoque y Diseño de Investigación

Este trabajo adopta un **diseño experimental cuantitativo de medidas repetidas**. Se manipulan sistemáticamente variables independientes (familias algorítmicas, grado de cooperación vecinal ρ y topologías de demanda vial) en entornos computacionales controlados para medir sus efectos sobre variables dependientes primarias vinculadas al desempeño operacional del tránsito, la equidad del servicio y la sostenibilidad ambiental.

La utilización de la simulación microscópica como plataforma experimental primaria responde a tres fundamentos científicos. El simulador utilizado es SUMO [ALBBW⁺18]:

1. **Seguridad y Viabilidad Operacional:** Evaluar algoritmos estocásticos de aprendizaje en tiempo real sobre una red urbana física real implica un riesgo inaceptable para la seguridad vial y la fluidez del tránsito vehicular.

2. **Aislamiento de Variables Exógenas:** La simulación permite neutralizar perturbaciones estocásticas no controladas (condiciones meteorológicas adversas, siniestros viales o fluctuaciones exógenas fuera del modelo), garantizando la validez interna del estudio.
3. **Replicabilidad Determinista:** Permite someter a todas las familias de control (algoritmos independientes, coordinados y de tiempo fijo) exactamente a las mismas condiciones de demanda mediante la fijación determinista de semillas pseudoaleatorias.

Para evaluar rigurosamente el impacto de las políticas aprendidas, se formalizan cuatro variables dependientes primarias a partir de la telemetría microscópica de SUMO (TripInfo):

- **Demora Acumulada Global (W_{net}):** Métrica operacional primaria expresada en segundos acumulados de retraso que experimentan los vehículos detenidos (velocidad $v < 0.1$ m/s) a lo largo de la red durante la ventana controlada:

$$W_{\text{net}} = \sum_{t=1}^T \sum_{v \in \mathcal{V}_t} w_v(t) \quad (3.1)$$

donde $\mathcal{V}_t$ es el conjunto de vehículos activos en la calzada en el paso temporal t y $w_v(t)$ es el tiempo de espera acumulado del vehículo v .

- **Tasa de Completitud Corregida (CR):** Proporción de la demanda generada que completa su itinerario dentro del horizonte de evaluación. La métrica también contabiliza los vehículos retenidos en los accesos de origen por sobresaturación:

$$\text{CR} = \frac{\text{throughput}}{\text{departed} + \text{pending_vehicles_end}} \quad (3.2)$$

donde throughput es la cantidad de vehículos completados y departed, la cantidad ingresada a la calzada. Además, pending_vehicles_end representa los vehículos retenidos. Se registran también la tasa de inserción ($\text{IR} = \frac{\text{inserted}}{\text{loaded}}$) y la completitud de insertados ($\text{CI} = \frac{\text{throughput}}{\text{inserted}}$).

- **Equidad Espacial de Demoras (Coeficiente de Gini, G_{wait}):** Cuantifica el grado de desigualdad en la distribución del tiempo de espera medio entre las distintas intersecciones semaforizadas de la red:

$$G_{\text{wait}} = \frac{2 \sum_{i=1}^N i \cdot \bar{w}_{(i)} - (N+1) \sum_{i=1}^N \bar{w}_{(i)}}{N \sum_{i=1}^N \bar{w}_{(i)}} \quad (3.3)$$

donde $\bar{w}_{(1)} \leq \bar{w}_{(2)} \leq \dots \leq \bar{w}_{(N)}$ son los tiempos de espera medios ordenados de cada controlador y N es el total de intersecciones. Esta métrica se complementa con la

desviación estándar inter-intersección (σ_{wait}) para evaluar si las políticas aprendidas distribuyen equitativamente el servicio o si sacrifican accesos secundarios en favor de arterias dominantes.

- **Emisiones Contaminantes de CO₂:** Masa total de dióxido de carbono emitida en gramos, calculada a partir de los perfiles instantáneos de aceleración y velocidad mediante los modelos microscópicos integrados en SUMO (PHEMlight/HBEFA [DLR25]).

La Figura 3.1 ilustra la secuencia metodológica estructurada en siete fases interconectadas que rige esta investigación.

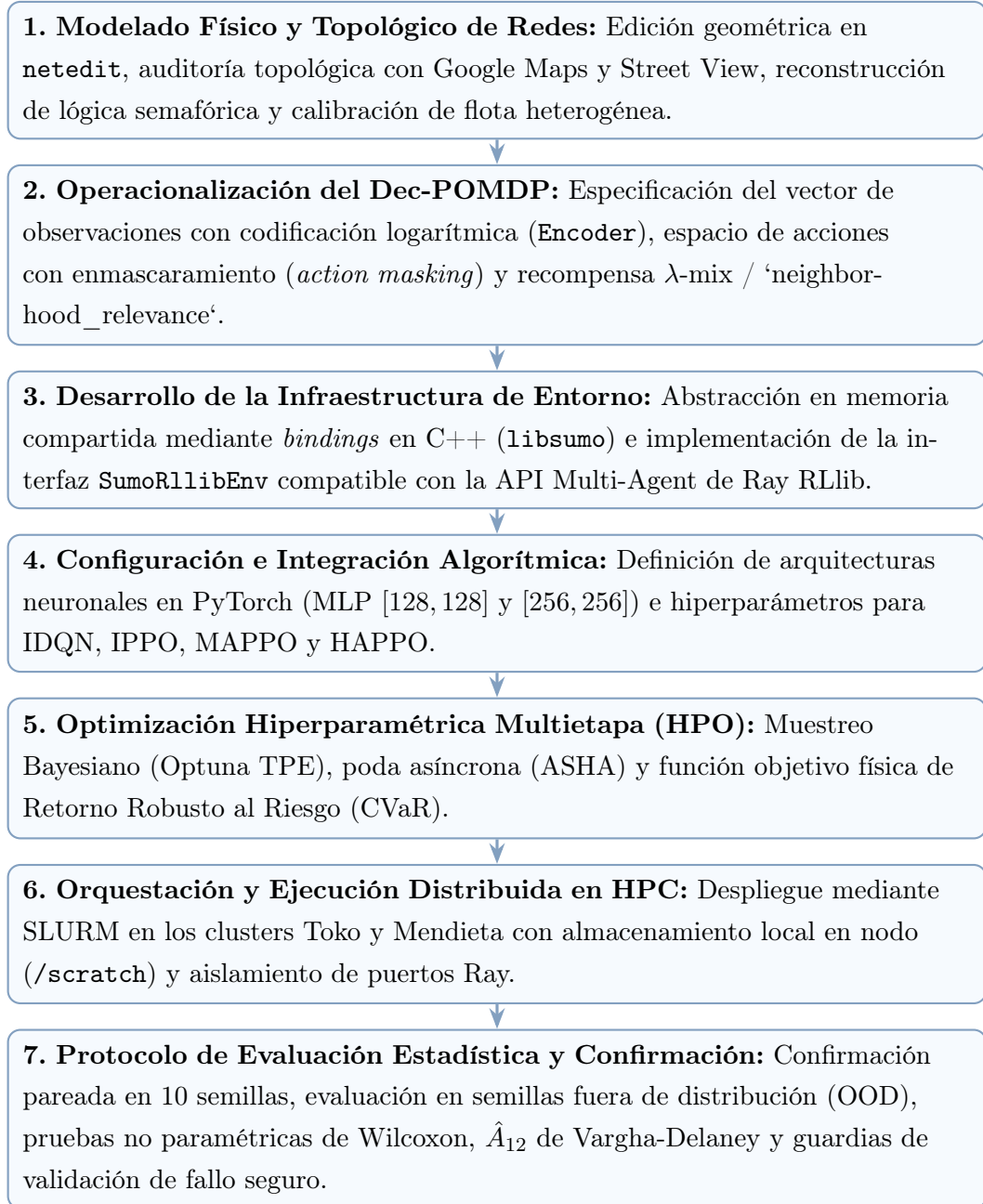


Figura 3.1: Flujo metodológico secuencial de la investigación experimental.

3.2 Especificación del Ecosistema Tecnológico

Para garantizar la reproducibilidad científica, la Tabla 3.1 detalla las librerías, marcos de trabajo y plataformas de cómputo sobre las cuales se desarrolló y ejecutó el entorno `marlTLC`.

Tabla 3.1: Especificación de las tecnologías y herramientas utilizadas en la investigación.

Componente	Herramienta / Versión	Función Metodológica
Lenguaje de Programación	Python 3.10+	Núcleo del sistema, tipado estático (<code>typing.Protocol</code>) y scripts de orquestación.
Simulador de Tránsito	SUMO 1.20+ / TraCI	Simulación microscópica, dinámica cinemática y telemetría de red.
<i>Bindings</i> de Simulación	<code>libsumo</code> (C++)	Ejecución de la simulación en memoria compartida mediante <i>bindings</i> en C++ (<code>LIBSUMO_AS_TRACI=1</code>).
Framework MARL	Ray RLlib 2.x [LLN ⁺ 18]	Abstracción de entornos multi-agente (<code>MultiAgentEnv</code> , <code>RLModuleSpec</code> , <code>LearnerGroup</code>).
Aprendizaje Profundo	PyTorch 2.x	Modelado neuronal y optimización (Actor-Crítico MLP, <i>action masking</i>).
Optimización Automática	Optuna 3.x [ASY ⁺ 19]	Muestreo Bayesiano TPE y poda asíncrona de pruebas (ASHA [LJR ⁺ 20]).
Infraestructura HPC	Clusters Toko y Mendieta	Cómputo distribuido multinúcleo gestionado vía SLURM con I/O local (<code>/scratch</code>).

La interacción técnica entre estas capas opera de forma integrada: Python orquesta la ejecución; Ray RLlib gestiona los procesos concurrentes de recolección de trayectorias (*env runners*); PyTorch computa las derivadas y optimiza las redes neuronales en los módulos de aprendizaje (*learners*); `SumoRllibEnv` encapsula la lógica episódica; y `libsumo` ejecuta los cálculos físicos de cinemática vehicular directamente en la memoria del proceso sin latencia de red ni serialización TCP.

3.3 Diseño y Calibración de Escenarios de Tránsito en SUMO

La evaluación empírica de los algoritmos se realiza sobre cuatro escenarios de red en SUMO, contruidos mediante la herramienta gráfica `netedit`, utilitarios de conversión `netconvert` y scripts de generación en Python, cubriendo un espectro representativo de complejidades topológicas y dinámicas de flujo.

3.3.1 Modelado Geométrico y Sanitización Topológica

La construcción de las redes de simulación se rige por un protocolo riguroso de diseño y validación en SUMO:

1. **Diseño Geométrico y Accesos:** Definición de calzadas, cantidad de carriles por aproximación y longitudes de almacenamiento en `netedit` o a partir de la importación de datos cartográficos desde la plataforma web de OpenStreetMap.
2. **Auditoría y Corrección Topológica con Datos de Terreno:** Dado que las fuentes cartográficas abiertas pueden contener inconsistencias operacionales, se realiza una verificación y corrección sistemática de los sentidos de circulación de las calles, la cantidad de carriles por calzada, las conexiones de giro y las maniobras prohibidas, contrastando directamente contra imágenes satelitales y vistas de calle de Google Maps y Google Street View.
3. **Sanitización y Programación Semafórica:** Las configuraciones de los semáforos estáticos corresponden a los planes generados por defecto por SUMO (`netconvert -tls.rebuild`) al compilar la red. Las señalizaciones, programas de fases y tiempos de ciclo originales se mantienen inalterados como línea base canónica, recalculando automáticamente la correspondencia entre enlaces físicos e índices de control semafórico (*linkIndex*) para garantizar la consistencia topológica.

3.3.2 Escenarios Sintéticos: 3×1 , 3×3 y 4×4 _H

Todas las redes sintéticas comparten reglas operacionales estandarizadas:

- **Límite de Velocidad Máxima:** 13.89 m/s (50 km/h) en todas las arterias urbanas.
- **Calibración de Flota Vehicular Heterogénea:** Para modelar el comportamiento urbano realista y evitar sesgos de homogeneidad, se definieron tres clases de vehículos en el archivo `urban_vehicle_mix.add.xml`, fundamentadas en modelos cinemáticos de confort fisiológico urbano [DLR25]:

1. **SLOWVEHTYPE (25%):** Conductores cautelosos o vehículos pesados con menor aceleración ($a = 1.9 \text{ m/s}^2$, $d = 4.0 \text{ m/s}^2$, $v_{\max} = 8.33 \text{ m/s} = 30 \text{ km/h}$).

2. **MEDVEHTYPE (60%)**: Vehículo de pasajeros promedio representativo. Parámetros: $a = 2.6 \text{ m/s}^2$, $d = 4.5 \text{ m/s}^2$ y $v_{\max} = 13.89 \text{ m/s} = 50 \text{ km/h}$.
3. **FASTVEHTYPE (15%)**: Conductores dinámicos con mayor aceleración desde reposo ($a = 3.1 \text{ m/s}^2$, $d = 5.0 \text{ m/s}^2$, $v_{\max} = 19.44 \text{ m/s} = 70 \text{ km/h}$, acotado a 50 km/h por el límite del carril).

Parámetros compartidos: longitud 4.5 m , brecha mínima de seguridad (minGap) 2.5 m e imperfección del conductor de Krauss $\sigma = 0.5$.

- **Generación de Demanda por Zonas (TAZ) y Arribos Estocásticos**: Las rutas se generan mediante la herramienta `random_routes_taz.py` a partir de matrices de zonas de asignación de tránsito (TAZ). Los intervalos entre llegadas vehiculares se modelan como procesos de Poisson con distribución exponencial:

$$\text{period} = \exp(\lambda_{\text{arr}}) \quad \text{con} \quad \lambda_{\text{arr}} = \frac{\text{flow}}{3600} \text{ veh/s} \quad (3.4)$$

- **Calibración por Bisección hacia Tasa de Completitud Saturada ($\text{CR} \approx 0.70$)**: Para garantizar una base de comparación homogénea y equitativa donde las políticas de aprendizaje deban operar en un régimen de saturación estandarizado, los flujos totales de cada red se calibraron mediante el método numérico de bisección bajo control semafórico de tiempo fijo sobre un horizonte efectivo de evaluación ($4200 \text{ s} = 600 \text{ s}$ de calentamiento $+3600 \text{ s}$ de control).

Las tres topologías sintéticas cumplen funciones analíticas complementarias:

- **Corredor Arterial 3×1 (3.260 veh/h)**: Tres intersecciones dispuestas en línea recta con flujo bidireccional dominante (40% este-oeste, 35% oeste-este). Constituye un Grafo Dirigido Acíclico (DAG) sin lazos de realimentación circular. La dinámica de desbordamiento y propagación de colas (*spillback*) es estrictamente lineal y la disipación de la congestión ocurre de forma natural aguas abajo, lo que permite evaluar la capacidad de sincronización de olas verdes sin interferencia de cuellos de botella circulares.
- **Malla Simétrica 3×3 (7.087 veh/h)**: Nueve intersecciones organizadas en una cuadrícula cerrada con demandas diagonales cruzadas (37.5% en cada eje). La alta interconexión genera lazos de dependencia circular: la saturación de una intersección central bloquea a sus vecinas, las cuales a su vez retienen el flujo de acceso, desencadenando una cascada de desbordamiento de colas (*spillback*) que culmina en un bloqueo mutuo total e irreversible de la red (*gridlock*). Constituye el banco de pruebas crítico para evaluar la estabilidad de MARL frente a estados absorbentes de congestión.

- **Cuadrícula 4×4 _H (7.202 veh/h):** Dieciséis intersecciones asimétricas con dos arterias principales de mayor capacidad. Estas arterias actúan como vías de drenaje que canalizan el tránsito de los nodos interiores hacia los extremos, ofreciendo rutas alternativas y mitigando los ciclos de bloqueo.

3.3.3 Escenario Urbano Real: Microcentro de la Ciudad de Mendoza

Para contrastar la validez y transferibilidad de las políticas aprendidas frente a condiciones de operación viales reales, se construyó un escenario del microcentro de la Ciudad de Mendoza a partir de su infraestructura topológica y de series temporales continuas de aforo vehicular captadas mediante cámaras termográficas.

Posición Metodológica: Escenario Restringido por Datos

Desde una perspectiva epistemológica y de modelado en ingeniería de transporte, el escenario de Mendoza se formaliza como un **escenario sintético empíricamente restringido por datos** (*data-constrained synthetic scenario*), diferenciándose conceptualmente de un gemelo digital irrestricto.

La inferencia de matrices Origen-Destino (OD) completas en redes urbanas a partir de mediciones volumétricas discretas en un conjunto reducido de puntos de control constituye un **problema inverso severamente subdeterminado e intrínsecamente mal condicionado** [CyMR18, PW16]. Para una red con $|\mathcal{Z}| = 37$ zonas de asignación de tránsito (TAZ), existen $|\mathcal{Z}|(|\mathcal{Z}| - 1) = 1.332$ pares direccionales potenciales (de los cuales 520 resultan físicamente alcanzables), frente a un número acotado de enlaces instrumentados con detectores en el acceso oriental. En tales condiciones, no existe una asignación OD única que satisfaga exclusivamente las observaciones empíricas.

Por consiguiente, este trabajo implementa una metodología estructurada de dos niveles:

1. **Restricción Dinámica Empírica:** Las mediciones continuas de aforo se emplean para gobernar el perfil temporal horario de la demanda total y fijar el flujo volumétrico observado en el corredor principal de ingreso.
2. **Estructuración Espacial Híbrida:** La distribución espacial de los viajes se formula combinando un conocimiento previo (*prior*) determinista basado en la jerarquía vial metropolitana con un remanente estocástico controlado sobre el espacio de pares alcanzables.

Adquisición, Auditoría y Preprocesamiento de Telemetría Empírica

La base empírica de aforo proviene de registros continuos de sensores de conteo y clasificación vehicular (cámaras térmicas Tecnotrans / FLIR ThermICam) con resolución de 15 minutos en el período Octubre 2025 – Abril 2026.

Si bien en fases preliminares se dispuso de información de cuatro dispositivos en el entorno del nudo de Costanera y Acceso Este, el relevamiento geoespacial determinó que las cámaras 192.168.1.167 (egreso hacia el este), 192.168.1.168 (circulación hacia el norte por Costanera) y 192.168.1.170 (circulación hacia el sur por Costanera) se emplazan entre 600 m y 800 m al este del perímetro de la red urbana modelada, midiendo flujos sobre la autopista que quedan fuera del modelo vial urbano. Por este motivo, **en la generación y calibración definitiva del escenario se utiliza exclusivamente la Cámara 169** (192.168.1.169), ubicada en el viaducto sobre el canal Cacique Guaymalén, la cual registra el ingreso vehicular hacia el oeste por la Avenida José Vicente Zapata hacia el microcentro.

Para garantizar la integridad y calidad del dataset de la Cámara 169, se aplicó un pipeline de auditoría y saneamiento de datos estructurado en cinco etapas:

1. **Filtrado Temporal y Estructural:** Descarte de marcas temporales corruptas (incluyendo fechas anómalas de época Unix 1970-01-01) o fuera del rango válido [2020–2035].
2. **Validación de Zonas y Clases Vehiculares:** Admisión estricta de zonas válidas ($\text{ZoneID} \in [1, 19]$) y clasificación física conforme a normativas de transporte: $\text{ClassID} \in \{1, 2, 3\}$, correspondiente a vehículos livianos ($l \leq 5$ m), medianos ($5 \text{ m} < l \leq 10$ m) y pesados ($l > 10$ m).
3. **Deduplicación por Clave Lógica:** Purga de registros redundantes mediante la clave única (DateUTC, ZoneID, ClassID), aplicando la regla determinista de conservación del último snapshot (`keep='last'`).
4. **Alineación Temporal (*Temporal Snapping*):** Reasignación de marcas de tiempo con desviaciones menores $|\Delta t| \leq 5$ s al cuarto de hora canónico más cercano y descarte de registros desalineados (*off-grid*).
5. **Filtro de Días Hábiles Completos:** Para evitar que interrupciones en la transmisión de datos se confundan con bajas de demanda real, se seleccionaron exclusivamente días hábiles (lunes a viernes) con cobertura completa del 100% de los intervalos diarios de 15 minutos (superando el umbral de guardia de $\geq 95\%$).

Este protocolo seleccionó 33 días hábiles completos no perturbados, arrojando en la Cámara 169 un volumen empírico diario medio de $\mu_{\text{obs}} = 25.283,85$ veh/día con una desviación estándar interdiaria de $\sigma_{\text{obs}} = 2.316,54$ veh/día.

Mapeo Geoespacial, Corrección Topológica y Señalización

El trazado geométrico base se obtuvo a partir de la exportación de datos de OpenStreetMap y su posterior conversión a formato de red SUMO (`.net.xml`). El dominio

modelado delimita el microcentro urbano de la Ciudad de Mendoza en el polígono comprendido entre Av. José Vicente Zapata / Colón (límite sur), Av. General San Martín (eje central norte-sur), Av. Manuel Belgrano (límite oeste) y Av. Las Heras (límite norte), proyectado en coordenadas UTM Zona 19S (WGS84).

Proxy de Ingreso Este (TAZ 28): Dado que el nudo de Costanera se ubica al este del perímetro del modelo, la Cámara 169 se proyecta como el proxy de inyección este en el primer tramo interno disponible de la calzada de ingreso: el enlace (*edge*) 93904440#0, asignado al origen de la zona TAZ 28.

Auditoría y Corrección Topológica con Google Maps y Street View: Tras la conversión inicial desde OpenStreetMap, se detectaron discrepancias operacionales y geométricas respecto a la infraestructura física real. Por ello, se llevó a cabo un proceso exhaustivo de auditoría y corrección apoyado en imágenes satelitales de Google Maps y relevamiento visual mediante Google Street View:

- **Restricciones de Giro y Maniobras Prohibidas:** Se implementaron las restricciones de giro reglamentarias observadas en terreno, tales como la prohibición estricta de giro a la izquierda desde Av. General San Martín hacia el sur hacia Av. Colón, y se preservaron giros semaforizados permitidos (por ejemplo, desde calle Perú hacia Av. Las Heras).
- **Sentidos de Circulación y Conectividad Vial:** Se corrigieron los sentidos de circulación de calzadas secundarias que figuraban de forma incorrecta como bidireccionales o invertidas en OpenStreetMap, y se restituyó la continuidad física de los cuatro carriles de ingreso de la Av. José Vicente Zapata hacia la trama interior (93904440#0 a 93904440#1).
- **Cantidad de Carriles y Accesos Laterales:** Se ajustó el número real de carriles de aproximación por calzada y se habilitaron las conexiones de inserción en calles transversales de acceso (Catamarca, Buenos Aires y Leandro N. Alem).
- **Configuración de Semáforos Estáticos y Señalización:** Las configuraciones de los semáforos de tiempo fijo corresponden a los programas generados por defecto por SUMO (`-tls.rebuild`) al compilar la red. Las señalizaciones, secuencias de fases y duraciones de ciclo originales no fueron alteradas ni reprogramadas manualmente, preservándose intactas como la línea base canónica sobre la cual operan los semáforos de tiempo fijo y contra la cual se evalúan los controladores adaptativos MARL.

Modelado Dinámico de Demanda y Factor de Expansión Territorial

El flujo vehicular en cada paso temporal se modela a partir del perfil horario representativo $\bar{q}_{\text{obs},169}(h)$ ($h \in [0, 23]$), promediado sobre los 33 días hábiles completos. El análisis de distribución temporal revela que el período diurno de 12 horas (06 : 00–18 : 00) concentra

el 69,29% del flujo diario total, mientras que la ventana matutina de hora pico (06 : 30–08 : 30) concentra el 12,53%.

Para proyectar el aforo del enlace observado a la escala metropolitana completa de la red, se aplica un **Factor de Expansión Territorial Dinámico** [CyMR18, PW16] gobernado por el volumen diario total estimado $V_{\text{diario}} = 260.000$ veh/día (adoptado tras una exploración de sensibilidad en el rango 250.000–290.000 veh/día):

$$f_{\text{exp}} = \frac{V_{\text{diario}}}{\sum_{h=0}^{23} \bar{q}_{\text{obs},169}(h)} = \frac{260.000}{25.283,85} \approx 10,2832 \quad (3.5)$$

La demanda total de la red en la hora h queda determinada por:

$$Q_{\text{red}}(h) = \bar{q}_{\text{obs},169}(h) \cdot f_{\text{exp}} \quad (3.6)$$

Adoptando una tasa de retención de detector $\text{camera_retention} = 1,0$, reconociendo que la Cámara 169 captura el flujo de aproximación representativo del acceso oriental.

Estructuración Espacial de la Demanda: Contrato OD Canónico v14 (60/40)

A partir de las 37 zonas TAZ, se computaron 520 pares OD físicamente alcanzables mediante búsqueda en anchura (BFS) sobre el grafo de conectividad vial. La distribución de la demanda horaria $Q_{\text{red}}(h)$ se rige por el **Contrato OD Canónico v14**, formulado como una partición híbrida:

$$p_{od} = \begin{cases} p_{od}^{\text{calib}} & \text{si } (o, d) \in \mathcal{P}_{\text{exp}} \quad \text{con} \quad \sum_{(o,d) \in \mathcal{P}_{\text{exp}}} p_{od}^{\text{calib}} = 0,60 \\ 0 & \text{si } (o, d) \in \mathcal{Z}_{\text{struct}} \\ \frac{0,40}{|\mathcal{P}_{\text{libre}}|} \cdot \xi_{od} & \text{si } (o, d) \in \mathcal{P}_{\text{libre}} = \mathcal{P}_{\text{reach}} \setminus (\mathcal{P}_{\text{exp}} \cup \mathcal{Z}_{\text{struct}}) \end{cases} \quad (3.7)$$

donde $\mathcal{P}_{\text{exp}}$ es el conjunto de pares explícitos, $\mathcal{Z}_{\text{struct}}$ los ceros estructurales, $\mathcal{P}_{\text{libre}}$ los pares libres alcanzables y ξ_{od} factores de asignación estocástica.

Componente Fijo Calibrado (60%): Estructurado en 476 entradas en `mendoza_od_proportions.json` (473 pares con cuotas positivas y 3 ceros directos) para reflejar la jerarquía vial observada en Mendoza:

- **Jerarquía de Arterias Principales:** Priorización de los corredores Av. Vicente Zapata (TAZ 28), Av. General San Martín (TAZ 19/29), Av. Colón / Emilio Civit (TAZ 7), Av. Las Heras (TAZ 37/38) y Av. Manuel Belgrano (TAZ 0).
- **Intervención en TAZ 12 (Av. Perú):** Reducción dirigida del tráfico de origen en –20% teórico (desvío realizado de –0,47% respecto a la cuota objetivo de 0,03657). Esta intervención fue requerida para mitigar la sobrecarga inducida sobre calle San Lorenzo (*edge* 249618899), previniendo el colapso temprano de los giros de distribución.

- **Incremento de Recorridos Pasantes en Patricias Mendocinas:** Aumento en la cuota de pares longitudinales estables (+9,3% realizado respecto a la línea base histórica) para modelar adecuadamente la función de drenaje norte-sur de dicha vía comercial.

Ceros Estructurales (Z_{struct}): Pares OD alcanzables pero topológicamente frágiles (por ejemplo, $21 \rightarrow 17$, $0 \rightarrow 3$, $10 \rightarrow 5$, $21 \rightarrow 12$, $23 \rightarrow 12$, $26 \rightarrow 12$) reciben explícitamente una probabilidad de asignación $p_{od} = 0,0$. Esto impide que el remanente aleatorio asigne vehículos a maniobras que provoquen bloqueo mutuo (*gridlock*) o teletransportaciones por sobreflujo en accesos estrechos.

Remanente Estocástico (40%) y Selección de Semilla: El remanente se distribuye entre los pares alcanzables libres mediante `random_routes_taz.py`. Durante el proceso de diseño del escenario, se evaluaron 18 semillas pseudoaleatorias candidatas; 10 completaron la simulación continua de 12 horas sin colisiones ni teletransportaciones. Se seleccionó la semilla de remanente 104773 por presentar la mínima desviación frente a las cuotas objetivo de calibración, fijando la semilla de simulación SUMO en 104729. Una vez validada, la realización de demanda se congeló de forma inmutable (`mendoza_12h.rou.xml`), asegurando que todas las comparaciones algorítmicas se efectúen sobre exactamente el mismo conjunto determinista de viajes.

Dinámica Microscópica y Heterogeneidad de Flota: Las inserciones se programan como procesos de Poisson con distribución exponencial de intervalos entre llegadas (`period="exp(rate)"` con $\lambda_{od} = \frac{Q_{od}}{3600}$ veh/s), asignando la distribución de flota heterogénea urbana `td_urban_mix` (25% lentos, 60% estándar y 15% dinámicos).

Protocolo de Validación Operacional y Criterio GEH

El escenario canónico v14 se validó rigurosamente bajo dos horizontes temporales de evaluación, cuyas métricas de salud de red se sintetizan en la Tabla 3.2:

1. **Escenario Base Canónico (12 Horas, 06 : 00–18 : 00):** Horizonte experimental completo (43.200 s) que reproduce la evolución continua del tránsito diurno.
2. **Ventana Operacional Proxy (2 Horas, 06 : 30–08 : 30):** Recorte temporal de hora pico ($t = [1800, 9000]$ s) empleado para la exploración iterativa de controladores y evaluaciones computacionales de alta demanda.

Tabla 3.2: Métricas operacionales de validación del escenario canónico v14 de Mendoza.

Ventana Temporal	Cargados	Insertados	Completados	CR	Inserción	Compleitud	Teleports
2h (06 : 30–08 : 30)	30.629	23.735	22.702	0,7412	0,7749	0,9565	0
12h (06 : 00–18 : 00)	180.163	143.819	143.022	0,7938	0,7983	0,9945	0

Validación de Calidad del Flujo (Estadístico GEH): Conforme a las recomendaciones metodológicas para la calibración de demanda en simulación microscópica [DLR25, PW16], la fidelidad de la inyección en el corredor José Vicente Zapata (TAZ 28) frente a los aforos empíricos de la Cámara 169 se evaluó mediante el estadístico GEH:

$$\text{GEH} = \sqrt{\frac{2(M - C)^2}{M + C}} \quad (3.8)$$

donde M es el volumen modelado y C el conteo empírico observado. En todos los intervalos horarios evaluados durante las 12 horas de simulación, el estadístico arrojó valores $\text{GEH} \leq 0,121$ (con un valor en hora pico matutina de las 08:00 de $\text{GEH} = 0,075$), superando holgadamente el criterio estándar de aceptación internacional ($\text{GEH} < 5,0$ en más del 85% de los puntos de aforo).

3.4 Arquitectura de Integración Entorno-Agentes

El marco computacional `marlTLC` integra el simulador SUMO con la librería Ray RLlib a través de la interfaz personalizada `SumoRllibEnv`. La especificación detallada de las clases y patrones de diseño orientados a objetos que sustentan esta arquitectura se expone en el Apéndice A.

3.4.1 Ciclo Episódico e Interfaz de Simulación

Para maximizar la eficiencia en la recolección de muestras durante el entrenamiento distribuido, las simulaciones prescinden de la interfaz gráfica y se ejecutan mediante la abstracción nativa C++ `libsumo`. Esta integración se activa con la variable de entorno `LIBSUMO_AS_TRACI=1` y enlaza el simulador directamente en la memoria compartida del proceso Python, eliminando la serialización y la latencia de comunicación por sockets TCP de la API TraCI estándar.

El ciclo de vida episódico opera bajo el siguiente protocolo de inicialización y control:

1. **Período de Calentamiento Inicial (*warmup*):** Cada episodio ejecuta una fase inicial de 600 segundos bajo control semafórico de tiempo fijo. Esta fase puebla la red con vehículos hasta alcanzar el régimen estacionario saturado, evitando que los agentes aprendan sobre una red vacía no representativa.
2. **Almacenamiento y Restauración de Puntos de Control (*checkpoint*):** Al finalizar la fase de calentamiento, el estado microscópico completo de la red se almacena en memoria o archivo de estado XML (`state_pid...xml`). Cada invocación a la función `reset()` restaura directamente este punto de control, garantizando que el entrenamiento comience instantáneamente en condiciones de congestión realista sin incurrir en el costo computacional de resimular el período inicial.

El tiempo de control efectivo `sim_time` transcurre exclusivamente en el intervalo $[t_{\text{warmup}}, t_{\text{warmup}} + \text{sim_time}]$.

3. **Paso de Decisión y Transiciones Semafóricas:** Las decisiones se ejecutan en pasos discretos de $\Delta t = 5.0$ s. Cuando un agente determina un cambio en la fase activa, el entorno intercala de forma obligatoria la fase amarilla de seguridad de duración `yellow_time = 4` s, requiriendo además un tiempo mínimo de verde `min_green_time = 5` s antes de habilitar una nueva transición, garantizando el despeje de los vehículos en la zona de conflicto.

—

3.5 Instanciación del Modelo Dec-POMDP

La interacción distribuida entre las intersecciones semaforizadas se formaliza mediante la tupla Dec-POMDP instanciada en la librería `marlTLC`.

3.5.1 Espacio de Observaciones y Codificación Logarítmica

La observación local $o_i^{(t)}$ recibida por el agente i en el paso t combina la fase activa actual y el tiempo de espera acumulado por carril entrante (w_l), cuantizado mediante codificación logarítmica.

Para evitar que valores extremos de tiempo de espera distorsionen la escala del vector de entrada y provoquen la saturación de los gradientes en las redes neuronales, se aplica la clase `Encoder` (`src/marlTLC/utlis/encoder.py`). Esta transformación aplica una compresión logarítmica no lineal híbrida que proporciona alta resolución para demoras bajas y una pendiente lineal acotada para demoras elevadas:

$$e(x) = \text{ceil} \left(F \cdot \log_2 \left(\frac{x}{M \cdot L} + 1 \right) + L^2 \cdot x \right) \quad (3.9)$$

donde $L = \frac{I-1}{M}$ y $F = (1 - L) \frac{I-1}{\log_2(1/L+1)}$, mapeando el valor continuo x al rango discreto $[0, I - 1]$.

En esta formulación:

- **Número de Intervalos (I):** Corresponde al hiperparámetro `encoder_intervals` (explorado en el rango $[4, 64]$), que define la resolución del espacio discreto y es optimizado automáticamente mediante exploración Bayesiana en Optuna.
- **Cota de Saturación del Carril (M):** Corresponde a `max_lane_value`, determinado empíricamente calculando el percentil 99 (P_{99}) de los tiempos de espera acumulados en simulaciones preliminares de cada red: $M = 1184$ s para 3×1 , $M = 1136$ s para 3×3 , $M = 712$ s para $4 \times 4_H$ y $M = 1494$ s para el microcentro de la Ciudad de Mendoza.

3.5.2 Espacio de Acciones y Enmascaramiento Dinámico

El espacio de acciones $\mathcal{A}_i = \{0, 1, \dots, K_i - 1\}$ representa el conjunto de fases verdes seleccionables por la intersección i . En cruces con geometrías asimétricas o restricciones de maniobra, la capa personalizada `ActionMaskingTorchRLModule` aplica un enmascaramiento dinámico sobre los *logits* $z_i(a)$ antes del cómputo de la distribución Softmax:

$$\text{logit}_i(a) = \begin{cases} z_i(a) & \text{si la fase } a \text{ es válida y cumple } t_{\text{verde}} \geq \text{min_green_time} \\ -10^9 & \text{en caso contrario} \end{cases} \quad (3.10)$$

garantizando que el agente asigne probabilidad nula a combinaciones incompatibles y asegurando la integridad física de las fases.

3.5.3 Función de Recompensa Compuesta y Coordinación Vecinal

La función de recompensa resuelve el compromiso entre el desempeño operacional local y la cooperación distribuida sin recurrir a términos heurísticos adicionales.

1. **Recompensa Local Diferencial (L_i):** Cuantifica la variación en el tiempo de espera acumulado entre pasos temporales consecutivos:

$$L_i^{(t)} = W_{\text{acc},i}(t-1) - W_{\text{acc},i}(t) \quad (3.11)$$

2. **Recompensa Compartida Vecinal (N_i):** Corresponde al promedio de las señales locales obtenidas por las intersecciones pertenecientes al vecindario topológico directo $\mathcal{N}(i)$:

$$N_i^{(t)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} L_j^{(t)} \quad (3.12)$$

3. **Mezcla Convexa Normalizada:** Para desacoplar la escala de la recompensa del grado de cooperación, se adopta una combinación convexa gobernada por el hiperparámetro $\rho \in [0, 0.95]$:

$$R_i^{(t)} = (1 - \rho)L_i^{(t)} + \rho N_i^{(t)} \quad \text{con } \rho \in [0, 0.95] \quad (3.13)$$

donde $\rho = 0$ representa el ancla puramente local. Esta formulación acota la escala de la señal y permite controlar explícitamente el grado de cooperación; no garantiza por sí sola preservar el ordenamiento de recompensas ni las trayectorias de valor del componente local.

3.6 Implementación Algorítmica e Hiperparámetros

Los cuatro algoritmos evaluados (los esquemas independientes IDQN e IPPO, y las arquitecturas de entrenamiento centralizado MAPPO y HAPPO) se implementaron en PyTorch e integraron en la API de Ray RLlib mediante el módulo de fábrica `algo_factory.py`.

3.6.1 Mecanismos de Aprendizaje Independiente y Centralizado

- **IDQN e IPPO (Aprendizaje Independiente):** Cada intersección aprende una política local $\pi_{\theta_i}(a_i|o_i)$ tratando a los demás controladores como parte del entorno no estacionario. En IPPO, cada agente cuenta con su propia red de valor local $V_{\phi_i}(o_i)$.
- **MAPPO (PPO Multi-Agente Centralizado [YVV⁺22]):** Emplea el paradigma CTDE. Durante la ejecución, cada actor selecciona acciones de forma descentralizada a partir de su observación local o_i . Durante el entrenamiento, un crítico centralizado compartido (`SharedCriticTorchRLModule`) evalúa el estado global concatenado $s = (o_1, o_2, \dots, o_N)$ para estimar $V_\phi(s)$ y computar las ventajas GAE (λ_{GAE}).
- **HAPPO (PPO para Agentes Heterogéneos [KCW⁺22]):** Extiende MAPPO al aprendizaje en agentes con espacios de acción o roles heterogéneos mediante el **Lema de Descomposición de Ventajas Multi-Agente**. En cada época de entrenamiento, los agentes se actualizan de forma secuencial según un orden determinado mediante la probabilidad `greedy_selection_prob` ($0.0 \rightarrow$ orden aleatorio, $1.0 \rightarrow$ orden codicioso según la magnitud de ventaja $|\bar{A}_i|$, $(0, 1) \rightarrow$ orden semi-codicioso). Tras optimizar el actor i_m , se recalcula el ratio post-optimización:

$$\rho_{\text{post}}^{(i_m)} = \frac{\pi_{\theta_{k+1}}^{(i_m)}(a|o)}{\pi_{\theta_k}^{(i_m)}(a|o)} \quad (3.14)$$

y se propaga multiplicativamente sobre el factor de ventaja $M^{(i_j)} \leftarrow M^{(i_j)} \cdot \rho_{\text{post}}^{(i_m)}$ para todos los agentes restantes $j > m$. Esta propagación implementa el ajuste secuencial de HAPPO; la garantía de mejora monótona corresponde al procedimiento teórico de región de confianza bajo sus supuestos explícitos y no se transfiere automáticamente a HAPPO-clip.

3.7 Búsqueda de Hiperparámetros (HPO)

La exploración de hiperparámetros se ejecutó con Optuna [ASY⁺19], Ray Tune y el algoritmo de poda asíncrona ASHA [LJR⁺20], en dos fases secuenciales:

1. **Fase 1 (Exploración Bayesiana Amplia):** Muestreo mediante Estimadores de Densidad Parzen Estructurados por Árbol (TPE) sobre un presupuesto de 100 pruebas por combinación (ampliable a 150), evaluando las arquitecturas compacta (`h128`: [128, 128]) y ancha (`h256`: [256, 256]) como estudios independientes de Ray Tune para evitar objetos no resueltos en `model_config`. El espacio de búsqueda comprende 13 dimensiones para MAPPO y 14 para HAPPO:

- $\rho \in [0.0, 0.95]$ (`uniform`)
- $\alpha \in [3 \times 10^{-5}, 10^{-3}]$ (`loguniform`)
- $\epsilon \in [0.10, 0.22]$ (`uniform`)
- $\lambda_{\text{GAE}} \in [0.88, 0.99]$ (`uniform`)
- $c_{\text{entropy}} \in [0.0, 0.01]$ (`uniform`)
- $\text{grad_clip} \in [0.25, 5.0]$ (`loguniform`)
- $c_{\text{KL}} \in [0.05, 0.5]$ (`loguniform`)
- $d_{\text{KL_target}} \in [0.005, 0.03]$ (`loguniform`)
- $\text{vf_clip} \in [5.0, 20.0]$ (`loguniform`)
- $K \in [5, 15]$ MAPPO / $[5, 18]$ HAPPO (`randint`)
- $N_{\text{episodes}} \in [2, 12]$ (`randint`)
- $I \in [4, 64]$ (`randint`)
- $B_{\text{mini}} \in \{128, 256, 512\}$ (`choice`)
- $p_{\text{greedy}} \in [0.20, 0.90]$ (solo HAPPO, `uniform`)

Parámetros fijos: $\gamma = 0.99$, `lane_metric = acc_waiting_time`, `use_kl_loss = True`. Semillas HPO de entrenamiento: 101 (3×1), 137 (3×3), 179 ($4 \times 4_H$); semillas de evaluación: 100101, 100137, 100179. La semilla histórica 42 queda prohibida.

2. **Fase 2 (Confirmación Pareada):** Reentrenamiento a presupuesto completo (40 iteraciones) sobre 10 semillas pareadas independientes de entrenamiento (211, 223, 227, 229, 233, 239, 241, 251, 257, 263) y 10 semillas de evaluación (200211, ..., 200263), comparando el ρ óptimo congelado frente a la línea base puramente local ($\rho = 0$).

Para prevenir la selección de políticas inestables que exhiban alto retorno promedio pero colapsos episódicos catastróficos por bloqueo total de la red (*gridlock*), la función objetivo optimiza el **Retorno Robusto al Riesgo** sobre las métricas físicas de SUMO TripInfo:

$$\text{stable_global_waiting} = \bar{W}_{\text{global}} + 1.0 \cdot \text{CVaR}_{0.2}(\text{positive_interiteration_increases}) \quad (3.15)$$

calculado sobre las últimas 10 iteraciones de entrenamiento (tras 5 iteraciones de calentamiento), penalizando explícitamente las caídas abruptas en la curva de aprendizaje.

3.7.1 Infraestructura de Cómputo y Orquestación

Dada la elevada dimensionalidad del espacio de búsqueda y el costo computacional de la simulación microscópica a paso fino en SUMO, la exploración y el entrenamiento final se desplegaron en los clusters de supercómputo **Toko** (Centro de Cómputo de Alto Rendimiento / CCAR - FCEFN / UNCuyo / CONICET) y **Mendieta** (CCAD - Universidad Nacional de Córdoba).

La orquestación distribuida fue gestionada mediante el sistema de colas SLURM aplicando los siguientes mecanismos de optimización y resiliencia:

- **Aislamiento de Procesos Ray en Nodo Único:** Configuración de instancias Ray de nodo único (`setup_ray_single_node`) con asignación dinámica de puertos basada en el identificador de trabajo:

$$\text{ray_port} = 20000 + (\text{SLURM_JOB_ID} \pmod{20000}) \quad (3.16)$$

y autodetección de dirección IP mediante `hostname -ip-address`, evitando colisiones de red entre múltiples trabajos concurrentes en el mismo nodo.

- **Gestión de Entrada/Salida en Almacenamiento Local (/scratch):** Para evitar saturar el ancho de banda del sistema de archivos distribuido en red (NFS) debido a la continua telemetría microscópica de SUMO (`tripinfo.xml`, `summary.xml`), las simulaciones y la recolección de trayectorias se ejecutaron en el almacenamiento volátil local de cada nodo de cómputo (`/scratch/$USER/ray_tmp/`).
- **Resiliencia y Persistencia ante Preempción:** Los scripts de envío incorporan trampas de señales POSIX (`trap ... EXIT USR1 TERM INT`) que ejecutan una sincronización incremental mediante `rsync` de los resultados y bases de datos SQLite de Optuna hacia el almacenamiento persistente `$HOME/ray_results/` antes de la finalización del trabajo o ante eventos de preempción, permitiendo la reanudación transparente mediante `Tuner.restore()`.

3.8 Evaluación de Rendimiento

El protocolo de evaluación final de los modelos entrenados se rigió por los siguientes criterios de rigor estadístico:

1. **Evaluación en Semillas Fuera de Distribución (OOD):** Para verificar la capacidad de generalización y descartar sobreajuste a secuencias de demanda partic-

ulares, las políticas finales se evaluaron con la configuración `final_evaluation` y las semillas 300101, 300137, 300179, 300191 y 300193.

2. **Pruebas de Hipótesis no Paramétricas y Tamaño del Efecto:** Las comparaciones de demora acumulada W_{net} entre algoritmos se evaluaron mediante la prueba de rangos con signo de Wilcoxon (con nivel de significancia $\alpha = 0.05$) y se complementaron con el estimador $\hat{A}_{12}$ de Vargha-Delaney y la Media Intercuartílica (IQM) con intervalos bootstrap del 95% para cuantificar la magnitud estandarizada del tamaño del efecto.
3. **Guardias de Validación de Fallo Seguro:** Se estableció el descarte automático de aquellas políticas candidatas que presentasen una reducción de vehículos completados (throughput) superior al 5%, un incremento en teletransportaciones vehiculares por colapso de carril o una degradación en la equidad de demoras (G_{wait}) mayor al 10% en comparación con el control de tiempo fijo.

4

Análisis y discusión de resultados

En este capítulo se presentan y analizan los resultados obtenidos de las ejecuciones de entrenamiento de los algoritmos de estudio mencionados. Con el objetivo de distinguir con claridad las etapas metodológicas del entrenamiento de los agentes, la sección se organiza en tres partes.

En primer lugar, se reportan los resultados del proceso de ajuste de hiperparámetros. En segundo lugar, se evalúan los modelos finales seleccionados en los escenarios planteados, desde redes sintéticas hasta el escenario ubicado en Ciudad de Mendoza. Finalmente, se desarrolla una discusión integradora de los hallazgos, sus implicancias y las principales limitaciones del estudio.

4.1 Resultados del ajuste de hiperparámetros

En esta sección se resume el proceso de ajuste de hiperparámetros y se justifica la configuración utilizada en la evaluación posterior de los modelos.

4.1.1 Diseño del proceso de ajuste

4.1.2 Resultados del ajuste y análisis de sensibilidad

4.1.3 Configuración seleccionada para la evaluación final

4.2 Resultados de los modelos finales

Esta sección reúne los resultados obtenidos con los modelos finales una vez fijada la configuración experimental. La exposición se organiza según el tipo de escenario analizado.

4.2.1 Criterios de evaluación y métricas de comparación

4.2.2 Desempeño en escenarios sintéticos: 3×1 , 3×3 , 4×4 heterogéneo

4.2.3 Desempeño en la red de carreteras de la Ciudad de Mendoza

4.3 Discusión e interpretación de resultados

En esta sección se integran los resultados presentados y se discuten sus principales implicancias en relación con los objetivos del trabajo.

5

Conclusiones

Por último las conclusiones.



Arquitectura de Software y Patrones de Diseño

En este apéndice se detalla la arquitectura de software del marco computacional `marlTLC`, sus especificaciones de diseño orientado a objetos, los diagramas de clases UML y la justificación de los patrones de diseño adoptados para desacoplar los componentes de simulación, aprendizaje por refuerzo y cálculo de recompensas.

A.1 Diseño Orientado a Objetos del Entorno de Simulación

La infraestructura de simulación se estructura siguiendo una separación estricta de responsabilidades. La clase abstracta `BaseSumoEnvironment` encapsula la inicialización de SUMO, el control temporal y la interacción de bajo nivel con la API `libsumo` / `traci`. A partir de ella, `SumoRllibEnv` adapta la simulación a la interfaz estándar `MultiAgentEnv` de Ray RLLib.

La Figura A.1 ilustra el diagrama de clases UML correspondiente a las entidades del entorno y su relación con los controladores semafóricos.

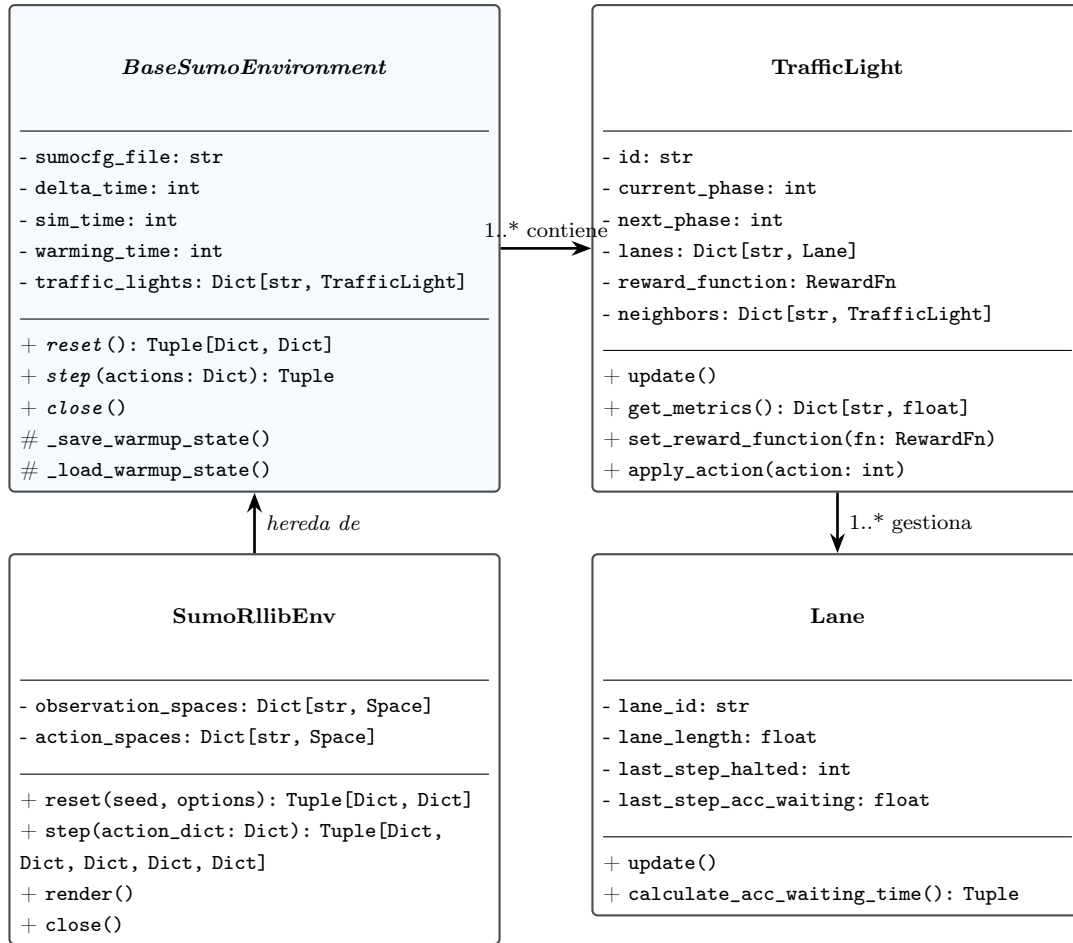


Figura A.1: Diagrama de clases UML de la infraestructura del entorno de simulación y controladores semafóricos.

A.2 Jerarquía de Funciones de Recompensa y Patrón Strategy

El cálculo de la recompensa se desacopla mediante el patrón **Strategy**, permitiendo intercambiar de forma transparente la métrica de optimización sin alterar el bucle principal de simulación.

La clase base abstracta **RewardFn** define la interfaz común `compute_reward(tl_id)`. Las clases concretas implementan formulaciones locales basadas en demoras acumuladas (**DiffAccWaitingTimeReward**), longitud de colas (**NegativeQueueLengthReward**) o presión de red (**PressureReward**). La coordinación distribuida se logra mediante el patrón **Composite**, implementado en **NeighborhoodRelevanceReward**, el cual combina de manera convexa la señal local y la señal promedio de los semáforos adyacentes provista por **NeighborAvgReward**.

La Figura A.2 detalla la estructura de clases del módulo de recompensas.

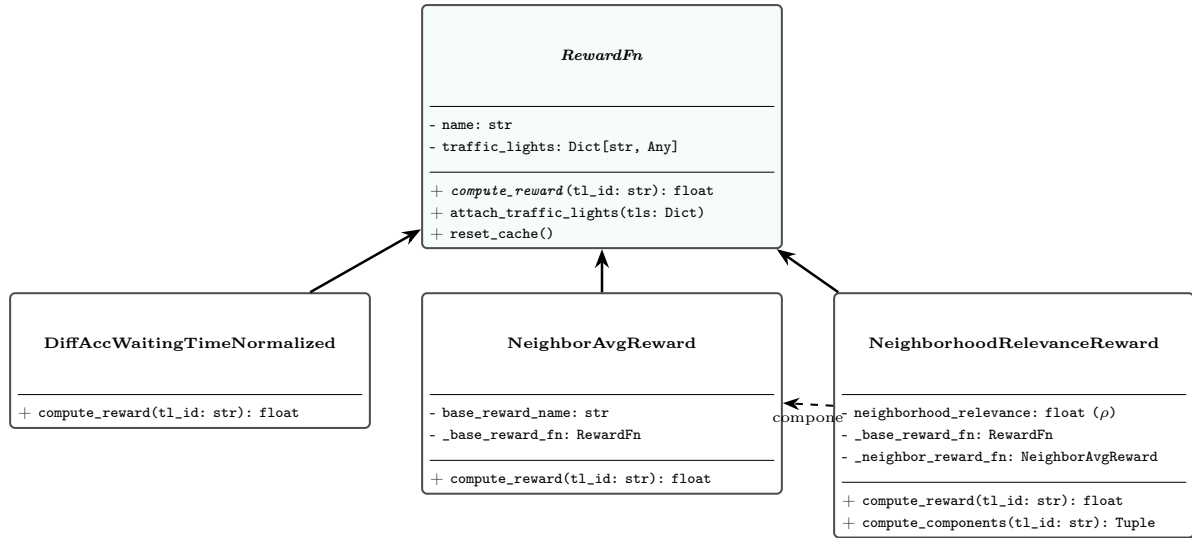


Figura A.2: Diagrama de clases UML de la jerarquía de funciones de recompensa (Patrones Strategy y Composite).

A.3 Patrones de Diseño Implementados

A lo largo del marco computacional marlTLC se aplican los siguientes patrones de diseño de software reconocidos:

1. Patrón Strategy (Estrategia):

- *Propósito:* Encapsular algoritmos de recompensa e intercambiarlos sin modificar los controladores semafóricos.
- *Implementación:* Interfaz base **RewardFn** y clases concretas de recompensa.

2. Patrón Factory Method y Registry (Fábrica con Registro Dinámico):

- *Propósito:* Instanciar dinámicamente funciones de recompensa y módulos neuronales a partir de cadenas de configuración o especificaciones compuestas (**RewardSpec**), permitiendo registrar nuevas funciones sin modificar la fábrica (Principio Abierto/Cerrado).
- *Implementación:* Clase **RewardFactory** y decorador **@register_reward**.

3. Patrón Composite (Objeto Compuesto):

- *Propósito:* Combinar recompensas locales y vecinales con una interfaz uniforme.
- *Implementación:* Clases de recompensa compuesta que agregan señales locales y vecinales.

4. Patrón Protocol / Structural Subtyping (Tipado Estructural):

- *Propósito:* Desacoplar el proveedor de métricas vecinales del tipo concreto `TrafficLight`, facilitando la prueba unitaria con objetos simulados (*mocks*) y evitando dependencias circulares.
- *Implementación:* Protocolo `NeighborInfoProvider` del entorno, con el decorador `@runtime_checkable`.

5. Patrón Adapter (Adaptador):

- *Propósito:* Traducir la API procedural y basada en comandos de TraCI / libsumo a las interfaces orientadas a objetos y vectorizadas de `MultiAgentEnv` (Ray RLLib) y `ParallelEnv` (PettingZoo).
- *Implementación:* `SumoRllibEnv` y `Observation`.

6. Patrón Decorator (Decorador):

- *Propósito:* Extender dinámicamente las capacidades de los módulos sin alterar su jerarquía de herencia.
- *Implementación:* Decoradores de registro `@register_reward` y envoltura de módulos de acción en PyTorch (`ActionMaskingTorchRLModule`).

Bibliografía

- [ACS24] Albrecht, Stefano V., Filippos Christianos y Lukas Schäfer: *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. <https://www.marl-book.com/>, Consultado en 2025.
- [ALBBW⁺18] Alvarez Lopez, Pablo, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner y Evamarie Wießner: *Microscopic Traffic Simulation using SUMO*. En *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, páginas 2575–2582. IEEE, 2018.
- [AOS20] Amato, Christopher, Frans A. Oliehoek y Matthijs T.J. Spaan: *A Survey of Decentralized Partially Observable Markov Decision Processes and Applications*. Journal of Artificial Intelligence Research, 69:1–72, 2020.
- [AS22] Ault, James y Guni Sharon: *Cooperative Multi-agent Reinforcement Learning Applied to Multi-intersection Traffic Signal Control*. En *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22)*, 2022. <https://people.engr.tamu.edu/guni/papers/CIKM22-MATCS.pdf>, Fuente: [7, 8] Destaca el "éxito limitado" de SARL y los enfoques independientes en redes.
- [ASY⁺19] Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takuya Ohta y Masanori Koyama: *Optuna: A next-generation hyperparameter optimization framework*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 2623–2631, 2019.
- [BB24] Bishop, Christopher M. y Hugh Bishop: *Deep Learning: Foundations and Concepts*. Springer, Cham, 2024.
- [CyMR18] Mayor R., Rafael Cal y: *Ingeniería de tránsito: Fundamentos y aplicaciones*. Alfaomega, 9na edición, 2018.
- [DLR25] DLR – Institute of Transportation Systems: *SUMO User Documentation*, 2025. <https://sumo.dlr.de/docs/>, Consultado en 2025.

- [dWGM⁺20] Witt, Christian Schroeder de, Tarun Gupta, Denys Makoviichuk, Viktor Maksimov, Edward Ivanov, Michael Schmid, Gregory Yakovlev, Philip Torr y cols.: *Is independent learning all you need in the starcraft multi-agent challenge?* arXiv preprint arXiv:2011.09533, 2020.
- [FA11] Fernández A., Rodrigo: *Elementos de la teoría del tráfico vehicular*. Fondo Editorial de la Pontificia Universidad Católica del Perú, 2011.
- [Fed08] Federal Highway Administration: *Traffic Signal Timing Manual*. Informe técnico FHWA-HOP-08-024, Federal Highway Administration, Office of Operations, 2008. Capítulo 4: Traffic Signal Design. Disponible en: <https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter4.htm>.
- [FH21] Feriani, Amal y Ekram Hossain: *Multi-Agent Reinforcement Learning: Key Differences from Single-Agent RL*. arXiv preprint arXiv:2508.07001, 2021. Fuente: [9] Define la no estacionariedad como una diferencia clave de MARL vs SARL.
- [FNF⁺17] Foerster, Jakob, Nantas Nardelli, Gregory Farquhar, Philip H.S. Torr, Pushmeet Kohli y Shimon Whiteson: *Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, páginas 1146–1155. PMLR, 2017.
- [GBC16] Goodfellow, Ian, Yoshua Bengio y Aaron Courville: *Deep Learning*. MIT Press, Cambridge, MA, 2016. <https://www.deeplearningbook.org/>.
- [KCW⁺22] Kuba, Jakub Grudzien, Ruiqing Chen, Muning Wen, Ying Wen, Fuchun Sun, Jun Wang y Yaodong Yang: *Trust region policy optimisation in multi-agent reinforcement learning*. En *International Conference on Learning Representations (ICLR)*, 2022. <https://arxiv.org/abs/2109.11251>.
- [KL02] Kakade, Sham y John Langford: *Approximately optimal reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 267–274, 2002.
- [KT03] Konda, Vijay R. y John N. Tsitsiklis: *On Actor-Critic Algorithms*. SIAM Journal on Control and Optimization, 42(4):1143–1166, 2003.
- [LJR⁺20] Li, Liam, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzvi, Moritz Hardt, Benjamin Recht y Ameet Talwalkar: *A system for massively parallel hyperparameter tuning*. En *Proceedings of Machine Learning and Systems (MLSys)*, volumen 2, páginas 230–246, 2020.

- [LLN⁺18] Liang, Eric, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan y Ion Stoica: *RLlib: Abstractions for distributed reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 3053–3062. PMLR, 2018.
- [LWT⁺17] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel y Igor Mordatch: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. En *Advances in Neural Information Processing Systems (NeurIPS)*, páginas 6379–6390, 2017.
- [MKS⁺15] Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg y D. Hassabis: *Human-level control through deep reinforcement learning*. *Nature*, 518(7540):529–533, 2015. <https://doi.org/10.1038/nature14236>.
- [MMLK25] Michailidis, Panagiotis, Iakovos Michailidis, Charalampos Rafail Lazaridis y Elias Kosmatopoulos: *Traffic Signal Control via Reinforcement Learning: A Review on Applications and Innovations*. *Infrastructures*, 10(5):114, 2025. Fuente: [4, 5] Revisión que documenta la evolución de RL como framework para TSC.
- [PW16] Pande, Anurag y Brian Wolshon: *Traffic Engineering Handbook*. John Wiley & Sons, Inc., 7th edición, 2016.
- [RSDW⁺18] Rashid, Tabish, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster y Shimon Whiteson: *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, páginas 4295–4304. PMLR, 2018.
- [SB18] Sutton, R.S. y A.G. Barto: *Reinforcement learning: An introduction*. The MIT Press, Cambridge, MA, 2nd edición, 2018.
- [Sha53] Shapley, Lloyd S: *Stochastic games*. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.
- [SLA⁺15] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan y Philipp Moritz: *Trust region policy optimization*. En *International Conference on Machine Learning (ICML)*, páginas 1889–1897. PMLR, 2015.

- [SLG⁺18] Sunehag, Peter, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls y Thore Graepel: *Value-Decomposition Networks For Cooperative Multi-Agent Learning*. En *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, páginas 2085–2087. International Foundation for Autonomous Agents and Multiagent Systems, 2018.
- [SML⁺16] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan y Pieter Abbeel: *High-dimensional continuous control using generalized advantage estimation*. En *International Conference on Learning Representations (ICLR)*, 2016.
- [SWD⁺17] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov: *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347, 2017.
- [TMK⁺17] Tampuu, Ardi, Tambet Matiisen, Dorian Kodelja, Ilya Zauss, Kristjan Korjus, Juhan Aru, Jaan Birk y Raul Vicente: *Multiagent cooperation and competition with deep reinforcement learning*. PloS one, 12(4):e0172395, 2017.
- [W⁺19] Wei, H. y cols.: *A survey on traffic signal control methods*, 2019. Available at: <https://arxiv.org/abs/1904.08117>.
- [Wie00] Wiering, Marco A: *Multi-agent reinforcement learning for traffic light control*. Machine learning, 2000.
- [YVV⁺22] Yu, Chao, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen y Yi Wu: *The surprising effectiveness of ppo in cooperative multi-agent games*. Advances in Neural Information Processing Systems, 35:24611–24624, 2022.
- [ZLYZ23] Zhou, Mengdi, Xiaoteng Li, Yaodong Yang y Weinan Zhang: *A Survey on Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning*. Artificial Intelligence Review, 2023.